



المحاضرة الرابعة:

اتخاذ القرارات

Making Decisions

Topics for Lecture 4

- 4.1 Relational Operators
- 4.2 The if Statement
- 4.3 Expanding the if Statement
- 4.4 The if/else Statement
- 4.5 Nested if Statements
- 4.6 The if/else if Statement
- 4.7 The if Statement with Initialization
- 4.8 Flags
- 4.9 Logical Operators
- 4.10 Checking Numeric Ranges with Logical Operators
- 4.11 Menus
- 4.12 Focus on Software Engineering: Validating User Input
- 4.13 Comparing Characters and Strings
- 4.14 The Conditional Operator
- 4.15 The switch Statement
- 4.16 The switch Statement with Initialization
- 4.17 More about Blocks and Variable Scope

4.1 Relational Operators

المفهوم: تسمح لك عوامل المقارنة (Relational Operators) بمقارنة القيم الرقمية والحروف وتحديد ما إذا كانت قيمة ما أكبر من، أو أقل من، أو تساوي، أو لا تساوي قيمة أخرى. حتى الآن، البرامج التي كتبتها تتبع هذا النهج البسيط:

- جمع المدخلات من المستخدم.
 - إجراء عملية حسابية واحدة أو أكثر.
 - عرض النتائج على الشاشة.
- أجهزة الكمبيوتر جيدة في إجراء العمليات الحسابية، ولكنها أيضاً ماهرة جداً في مقارنة القيم لتحديد ما إذا كانت قيمة ما أكبر من، أو أقل من، أو تساوي قيمة أخرى. هذه الأنواع من العمليات مفيدة لمهام مثل فحص أرقام المبيعات، تحديد الربح والخسارة، التحقق من رقم للتأكد من أنه ضمن نطاق مقبول، والتحقق من صحة المدخلات التي يوفرها المستخدم.
- يتم مقارنة البيانات الرقمية في لغة C++ باستخدام عوامل المقارنة. كل عامل مقارنة يحدد ما إذا كانت هناك علاقة محددة بين قيمتين.

Table 4-1 lists all of C++'s relational operators.

Table 4-1 Relational Operators

Relational Operators	Meaning
>	Greater than
<	Less than
>=	Greater than or equal to
<=	Less than or equal to
==	Equal to
!=	Not equal to

Table 4-2 shows examples of several relational expressions that compare the variables x and y.

Table 4-2 Relational Expressions

Expression	What the Expression Means
x > y	Is x greater than y?
x < y	Is x less than y?
x >= y	Is x greater than or equal to y?
x <= y	Is x less than or equal to y?
x == y	Is x equal to y?
x != y	Is x not equal to y?



NOTE: All the relational operators have left-to-right associativity. Recall that associativity is the order in which an operator works with its operands.

• The Value of a Relationship

إنّ، كيف تُستخدم تعبيرات المقارنة في البرنامج؟ تذكر أن جميع التعبيرات لها قيمة. تُعرف تعبيرات المقارنة أيضًا باسم التعبيرات المنطقية (Boolean Expressions)، مما يعني أن قيمتها يمكن أن تكون إما صحيحة (true) أو خاطئة (false). إذا كانت قيمة `x` أكبر من قيمة `y`، فإن التعبير `x > y` سيكون صحيحًا، بينما التعبير `y == x` سيكون خاطئًا.

Table 4-3 shows other relational expressions and their true or false values.

Table 4-3 (Assume x is 10 and y is 7.)

Expression	Value
$x < y$	False, because x is not less than y.
$x > y$	True, because x is greater than y.
$x \geq y$	True, because x is greater than or equal to y.
$x \leq y$	False, because x is not less than or equal to y.
$y \neq x$	True, because y is not equal to x.

To illustrate this more fully, look at Program 4-1.

Program 4-1

```

1 // This program displays the values of true and false states.
2 #include <iostream>
3 using namespace std;
4
5 int main()
6 {
7     bool trueValue, falseValue;
8     int x = 5, y = 10;
9
10    trueValue = x < y;
11    falseValue = y == x;
12
13    cout << "True is " << trueValue << endl;
14    cout << "False is " << falseValue << endl;
15    return 0;
16 }
```

Program Output

```

True is 1
False is 0
```



NOTE: Relational expressions have a higher precedence than the assignment operator. In the statement

```
z = x < y;
```

the expression $x < y$ is evaluated first, then its value is assigned to z.

Table 4-4 (Assume x is 10, y is 7, and z, a, and b are ints or bools.)

Statement	Outcome
<code>z = x < y</code>	z is assigned 0 because x is not less than y.
<code>cout << (x > y);</code>	Displays 1 because x is greater than y.
<code>a = x >= y;</code>	a is assigned 1 because x is greater than or equal to y.
<code>cout << (x <= y);</code>	Displays 0 because x is not less than or equal to y.
<code>b = y != x;</code>	b is assigned 1 because y is not equal to x.

4.2 The if Statement

المفهوم: يمكن لتعليمة `if` أن تجعل تعليمات أخرى تنفذ فقط تحت شروط معينة.

قد تفكر في تعليمات البرنامج الإجرائي (Procedural Program) كخطوات فردية تقوم بها أثناء سيرك في طريق ما. للوصول إلى الوجهة، يجب أن تبدأ من البداية وتأخذ كل خطوة واحدة تلو الأخرى حتى تصل إلى الوجهة. البرامج التي كتبتها حتى الآن تشبه "مسارًا" محددًا لتنفيذ البرنامج.

يُطلق على النوع الموجود في الشكل 4-1 اسم هيكل التسلسل (Sequence Structure) لأن التعليمات تُنفذ بالتسلسل دون انحراف في اتجاه آخر. ومع ذلك، تحتاج العديد من البرامج إلى أكثر من مسار تنفيذ. تتطلب العديد من الخوارزميات أن ينفذ البرنامج بعض التعليمات فقط في ظل ظروف معينة. يمكن تحقيق ذلك باستخدام هيكل القرار (Decision Structure).

في أبسط أشكاله، يتم تنفيذ إجراء معين فقط في حالة وجود شرط محدد. إذا لم يكن الشرط موجودًا، لا يتم تنفيذ الإجراء. يوضح الرسم التخطيطي في الشكل 4-2 منطق هيكل القرار. يمثل رمز الماسة (Diamond) سؤالاً من نوع نعم/لا أو شرطاً من نوع صحيح/خاطئ. إذا كانت الإجابة على السؤال "نعم" (أو إذا كان الشرط صحيحاً)، يتبع تدفق البرنامج مساراً يؤدي إلى تنفيذ إجراء معين. إذا كانت الإجابة على السؤال "لا" (أو إذا كان الشرط خاطئاً)، يتبع تدفق البرنامج مساراً آخر يتخطى هذا الإجراء.

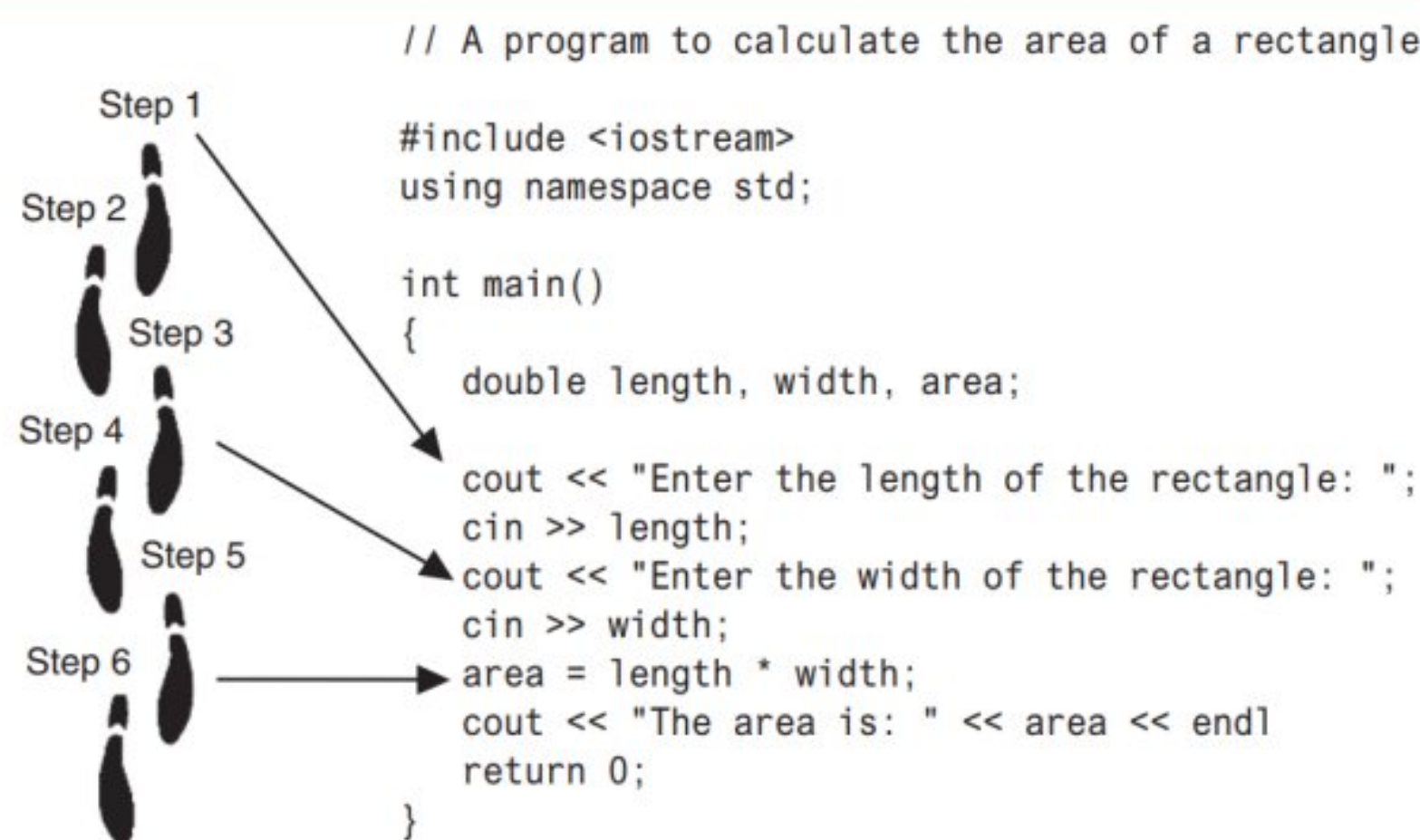
Figure 4-1 Sequence structure

Figure 4-2 Decision structure

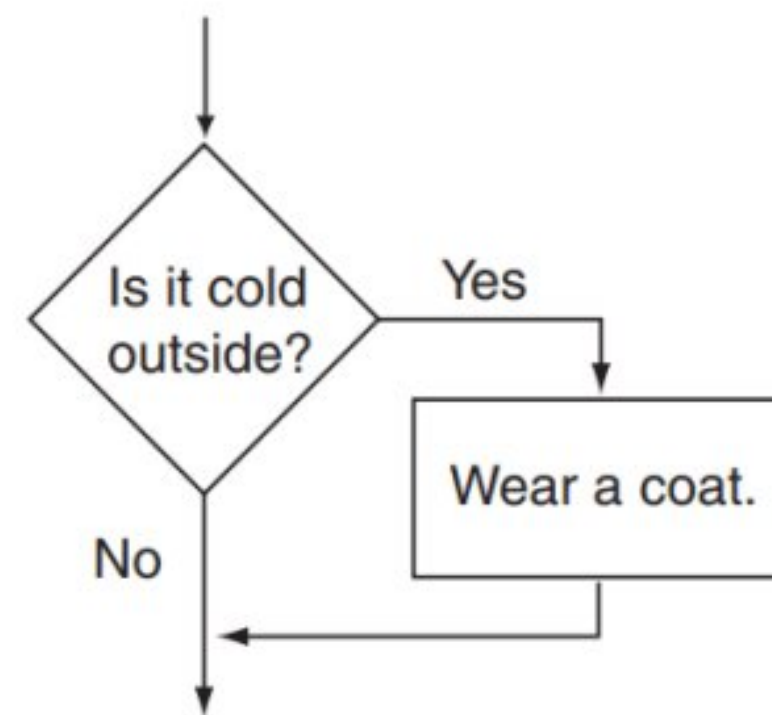
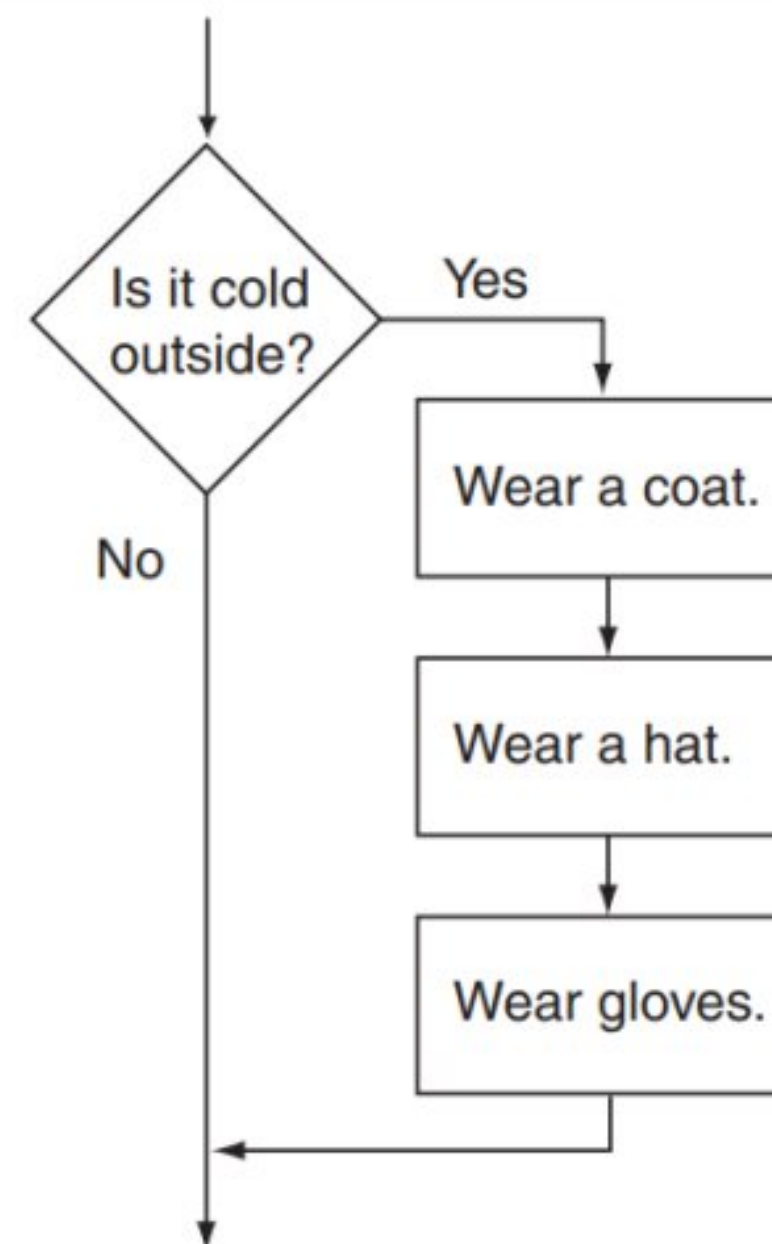


Figure 4-3 Decision structure



يوضح البرنامج 2-4 مثالاً على استخدام تعليمة `if`. يقوم المستخدم بإدخال ثلاث درجات اختبار، ويقوم البرنامج بحساب متوسطها. إذا كان المتوسط أكبر من 95، يهنئ البرنامج المستخدم على الحصول على درجة عالية.

Program 4-2

```
1 // This program averages three test scores
2 #include <iostream>
3 #include <iomanip>
4 using namespace std;
5
6 int main()
7 {
8     const int HIGH_SCORE = 95; // A high score is greater than 95
9     int score1, score2, score3; // To hold three test scores
10    double average; // To hold the average score
11
```



```

12 // Get the three test scores.
13 cout << "Enter 3 test scores and I will average them: ";
14 cin >> score1 >> score2 >> score3;
15
16 // Calculate and display the average score.
17 average = (score1 + score2 + score3) / 3.0;
18 cout << fixed << showpoint << setprecision(1);
19 cout << "Your average is " << average << endl;
20
21 // If the average is a high score, congratulate the user.
22 if (average > HIGH_SCORE)
23     cout << "Congratulations! That's a high score!\n";
24 return 0;
25 }

```

Program Output with Example Input Shown in Bold

Enter 3 test scores and I will average them: **80 90 70**
Your average is 80.0

Program Output with Different Example Input Shown in Bold

Enter 3 test scores and I will average them: **100 100 100**
Your average is 100.0
Congratulations! That's a high score!

Figure 4-4 if statement logic

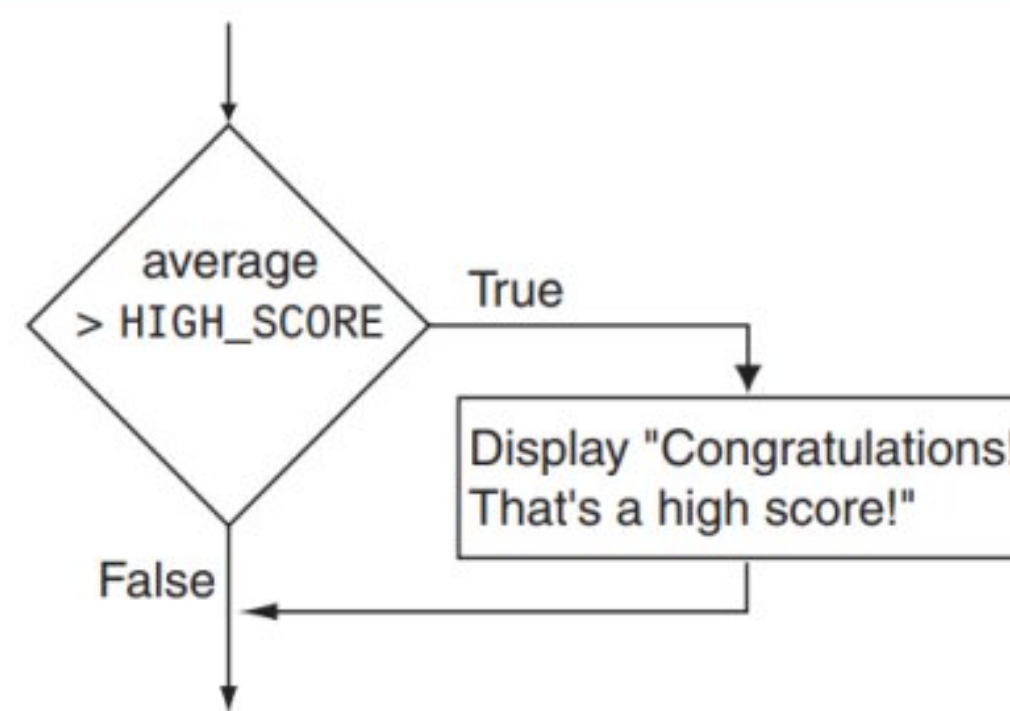


Table 4-5 if Statements and Their Outcomes

Statement	Outcome
if (hours > 40) overTime = true;	Assigns true to the bool variable overTime only if hours is greater than 40
if (value > 32) cout << "Invalid number\n";	Displays the message "Invalid number" only if value is greater than 32
if (overTime == true) payRate *= 2;	Multiplies payRate by 2 only if overTime is equal to true

- **Be Careful with Semicolons**

No semicolon goes here.

if (expression)
statement;

Semicolon goes here.

Program 4-3

```
1 // This program demonstrates how a misplaced semicolon
2 // prematurely terminates an if statement.
3 #include <iostream>
4 using namespace std;
5
6 int main()
7 {
8     int x = 0, y = 10;
9
10    cout << "x is " << x << " and y is " << y << endl;
11    if (x > y); // Error! Misplaced semicolon
12        cout << "x is greater than y\n"; //This is always executed.
13    return 0;
14 }
```

Program Output

```
x is 0 and y is 10
x is greater than y
```



NOTE: Indentation and spacing are for the human readers of a program, not the compiler. Even though the cout statement following the if statement in Program 4-3 is indented, the semicolon still terminates the if statement.

- **Comparing Floating-Point Numbers**

بسبب الطريقة التي يتم بها تخزين الأعداد ذات الفاصلة العائمة (floating-point numbers) في الذاكرة، تحدث أحياناً أخطاء في التقريب. هذا لأن بعض الأعداد الكسرية لا يمكن تمثيلها بدقة باستخدام النظام الثنائي. لذلك، يجب أن تكون حذراً عند استخدام عامل المساواة (==) لمقارنة الأعداد ذات الفاصلة العائمة. على سبيل المثال، يستخدم البرنامج 4-4 متغيرين من نوع 'double'، وهما 'a' و 'b'. يتم تهيئة كلا المتغيرين بالقيمة 1.5. ثم تتم إضافة القيمة 0.0000000000000001 إلى المتغير 'a'. من المفترض أن يجعل هذا محتويات المتغير 'a' مختلفة عن محتويات المتغير 'b'. ومع ذلك، وبسبب خطأ في التقريب، يظل المتغيران متساويين.

Program 4-4

```
1 // This program demonstrates how floating-point
2 // round-off errors can make equality operations unreliable.
3 #include <iostream>
4 using namespace std;
5
6 int main()
```

Program 4-4 (continued)

```
7 {
8     double a = 1.5;           // a is 1.5.
9     double b = 1.5;           // b is 1.5.
10
11     a += 0.0000000000000001; // Add a little to a.
12     if (a == b)
13         cout << "Both a and b are the same.\n";
14
15     if (a != b)
16         cout << "a and b are not the same.\n";
17
18     return 0;
19 }
```

Program Output

Both a and b are the same.

لتجنب المشكلات الناتجة عن أخطاء التقريب، يجب عليك الالتزام باستخدام عمليات المقارنة الأكبر من ('<') والأصغر من ('>') عند التعامل مع الأعداد ذات الفاصلة العائمة (floating-point numbers).

البرنامج 4-5 هو نسخة من البرنامج 2-4 تحاول التحقق من وجود متوسط مثالي يساوي 100. ومع ذلك، تم استخدام عامل الإسناد ('=') عن طريق الخطأ في تعليمة 'if'.

Program 4-5

```
1 // This program averages 3 test scores. The if statement
2 // uses the = operator, but the == operator was intended.
3 #include <iostream>
4 #include <iomanip>
5 using namespace std;
6
7 int main()
8 {
9     int score1, score2, score3; // To hold three test scores
10    double average;             // To hold the average score
11
12    // Get the three test scores.
13    cout << "Enter 3 test scores and I will average them: ";
14    cin >> score1 >> score2 >> score3;
15
16    // Calculate and display the average score.
17    average = (score1 + score2 + score3) / 3.0;
18    cout << fixed << showpoint << setprecision(1);
19    cout << "Your average is " << average << endl;
20
```

(program continues)

Program 4-5 (continued)

```
21 // Our intention is to congratulate the user
22 // for having a perfect score. But, this doesn't work.
23 if (average = 100) // WRONG! This is an assignment!
24     cout << "Congratulations! That's a perfect score!\n";
25 return 0;
26 }
```

Program Output with Example Input Shown in Bold

```
Enter three test scores and I will average them: 80 90 70 
Your average is 80.0
Congratulations! That's a perfect score!
```

4.3 Expanding the if Statement

المفهوم: يمكن لتعليمة `if` تنفيذ مجموعة من التعليمات بشكل مشروط عند وضعها داخل أقواس معقوفة `{}`.

ماذا لو كنت تريد أن تنفذ تعليمة `if` مجموعة من التعليمات وليس مجرد سطر واحد؟ على سبيل المثال، ماذا لو كان برنامج حساب المتوسط يحتاج إلى استخدام عدة تعليمات `cout`؟

```
if (expression)
{
    statement;
    statement;
    // Place as many statements here as necessary.
}
```


يوضح البرنامج 4-6، وهو تعديل آخر لبرنامج حساب المتوسط، هذا النوع من تعليمات `if`.

Program 4-6

```
1 // This program averages 3 test scores.
2 // It demonstrates an if statement executing
3 // a block of statements.
4 #include <iostream>
5 #include <iomanip>
6 using namespace std;
7
8 int main()
9 {
10     const int HIGH_SCORE = 95;           // A high score is 95 or greater
11     int score1, score2, score3;          // To hold three test scores
12     double average;                      // To hold the average score
13
14     // Get the three test scores.
15     cout << "Enter 3 test scores and I will average them: ";
16     cin >> score1 >> score2 >> score3;
17
18     // Calculate and display the average score.
19     average = (score1 + score2 + score3) / 3.0;
20     cout << fixed << showpoint << setprecision(1);
21     cout << "Your average is " << average << endl;
22
23     // If the average is high, congratulate the user.
24     if (average > HIGH_SCORE)
25     {
26         cout << "Congratulations!\n";
27         cout << "That's a high score.\n";
28         cout << "You deserve a pat on the back!\n";
29     }
30     return 0;
31 }
```

Program 4-6 (continued)

Program Output with Example Input Shown in Bold

Enter 3 test scores and I will average them: **100 100 100**
Your average is 100.0
Congratulations!
That's a high score.
You deserve a pat on the back!

Program Output with Different Example Input Shown in Bold

Enter 3 test scores and I will average them: **80 90 70**
Your average is 80.0



NOTE: Anytime your program has a block of code, all the statements inside the braces should be indented.

Don't Forget the Braces!

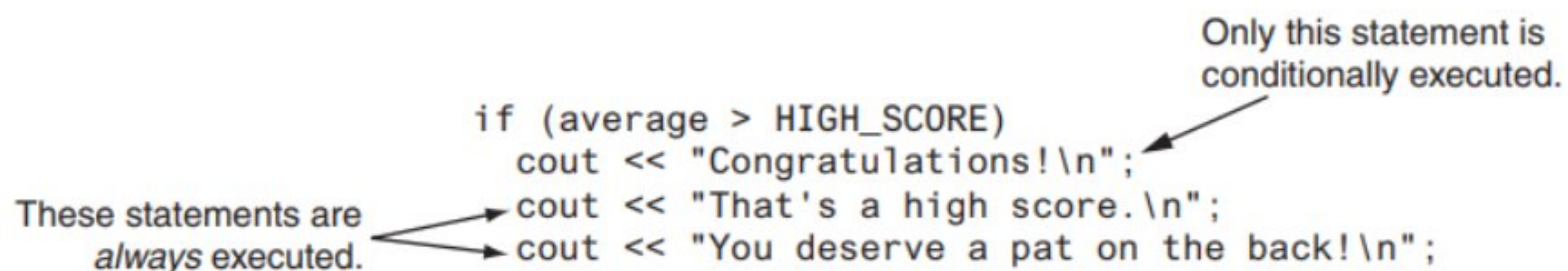
Program 4-7

```
1 // This program averages 3 test scores. The braces
2 // were inadvertently left out of the if statement.
3 #include <iostream>
4 #include <iomanip>
5 using namespace std;
6
7 int main()
8 {
9     const int HIGH_SCORE = 95;           // A high score is 95 or greater
10    int score1, score2, score3;          // To hold three test scores
11    double average;                      // To hold the average score
12
13    // Get the three test scores.
14    cout << "Enter 3 test scores and I will average them: ";
15    cin >> score1 >> score2 >> score3;
16
17    // Calculate and display the average score.
18    average = (score1 + score2 + score3) / 3.0;
19    cout << fixed << showpoint << setprecision(1);
20    cout << "Your average is " << average << endl;
21
22    // ERROR! This if statement is missing its braces!
23    if (average > HIGH_SCORE)
24        cout << "Congratulations!\n";
25        cout << "That's a high score.\n";
26        cout << "You deserve a pat on the back!\n";
27    return 0;
28 }
```

Program Output with Example Input Shown in Bold

```
Enter 3 test scores and I will average them: 80 90 70 Enter
Your average is 80
That's a high score.
You deserve a pat on the back!
```

Figure 4-5 Conditionally executed statement



4.4 The if/else Statement

المفهوم: تنفذ تعليمة `if/else` مجموعة من التعليمات إذا كان التعبير صحيحًا، أو مجموعة أخرى من التعليمات إذا كان التعبير خاطئًا. تُعتبر تعليمة `if/else` امتدادًا لتعليمة `if`. إليك شكلها العام:

```
if (expression)
    statement or block
else
    statement or block
```

Program 4-8 uses the if/else statement along with the modulus operator to determine if a number is odd or even.

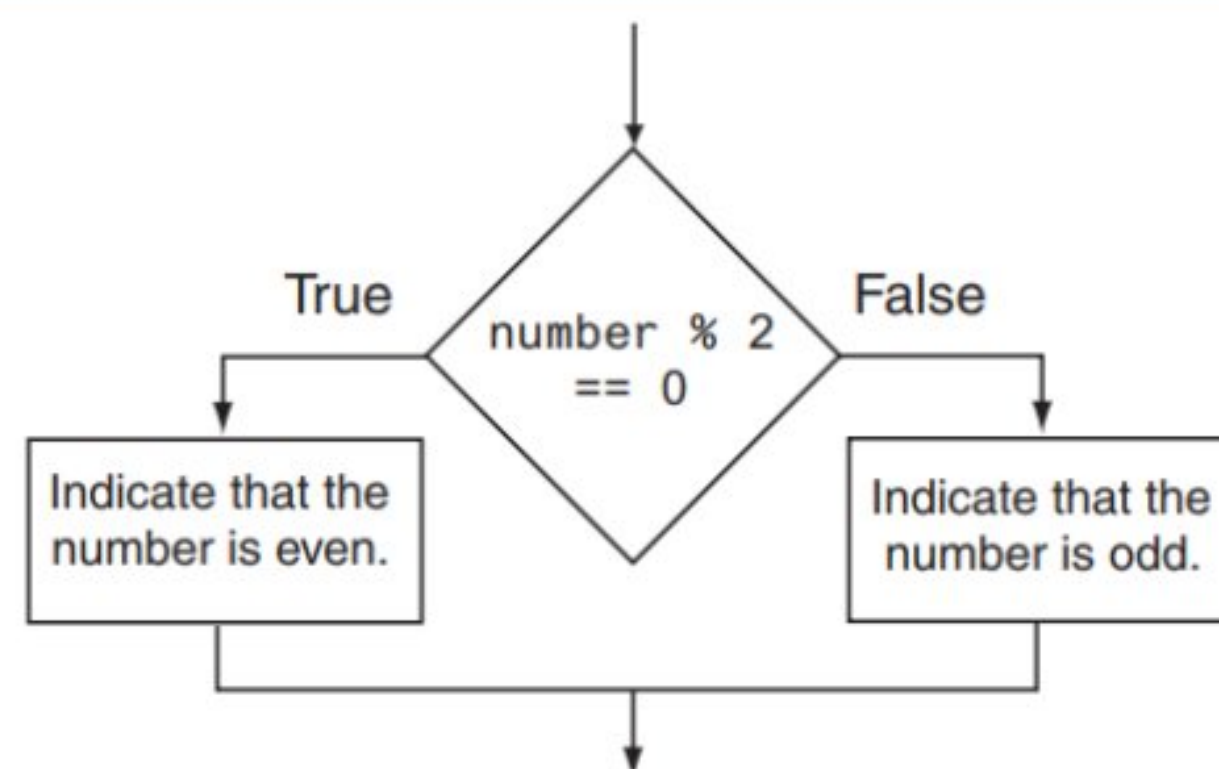
Program 4-8

```
1 // This program uses the modulus operator to determine
2 // if a number is odd or even. If the number is evenly divisible
3 // by 2, it is an even number. A remainder indicates it is odd.
4 #include <iostream>
5 using namespace std;
6
7 int main()
8 {
9     int number;
10
11     cout << "Enter an integer and I will tell you if it\n";
12     cout << "is odd or even. ";
13     cin >> number;
14     if (number % 2 == 0)
15         cout << number << " is even.\n";
16     else
17         cout << number << " is odd.\n";
18     return 0;
19 }
```

Program Output with Example Input Shown in Bold

Enter an integer and I will tell you if it
is odd or even. **17**
17 is odd.

Figure 4-6 if/else statement logic



القسمة على الصفر مستحيلة رياضياً، وعادةً ما تتسبب في تعطل البرنامج. هذا يعني أن البرنامج سيتوقف عن العمل قبل الأوان، وأحياناً مع ظهور رسالة خطأ. يُظهر البرنامج 4-9 طريقة لاختبار قيمة المقسوم عليه قبل إجراء عملية القسمة.

Program 4-9

```
1 // This program asks the user for two numbers, num1 and num2.
2 // num1 is divided by num2 and the result is displayed.
3 // Before the division operation, however, num2 is tested
4 // for the value 0. If it contains 0, the division does not
5 // take place.
6 #include <iostream>
7 using namespace std;
8
9 int main()
10 {
11     double num1, num2, quotient;
12
13     // Get the first number.
14     cout << "Enter a number: ";
15     cin >> num1;
16
17     // Get the second number.
18     cout << "Enter another number: ";
19     cin >> num2;
```

(program continues)

Program 4-9

(continued)

```
20
21 // If num2 is not zero, perform the division.
22 if (num2 == 0)
23 {
24     cout << "Division by zero is not possible.\n";
25     cout << "Please run the program again and enter\n";
26     cout << "a number other than zero.\n";
27 }
28 else
29 {
30     quotient = num1 / num2;
31     cout << "The quotient of " << num1 << " divided by ";
32     cout << num2 << " is " << quotient << ".\n";
33 }
34 return 0;
35 }
```

Program Output with Example Input Shown in Bold

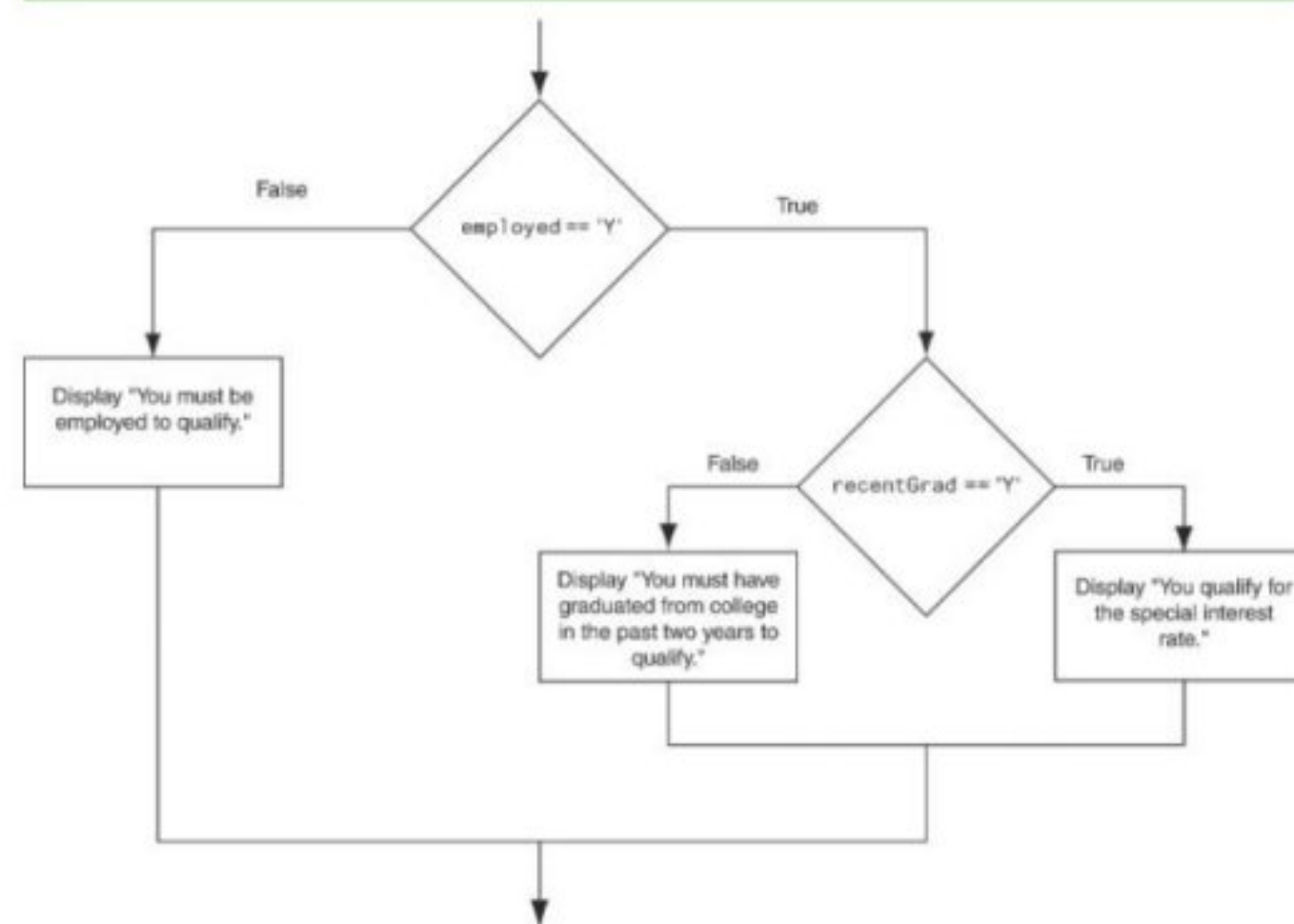
```
Enter a number: 10  Enter
Enter another number: 0  Enter
Division by zero is not possible.
Please run the program again and enter
a number other than zero.
```


4.5 Nested if Statements

المفهوم: لاختبار أكثر من شرط، يمكن وضع تعليمة `if` داخل تعليمة `if` أخرى (تداخل تعليمات `if`).

في بعض الأحيان، يجب وضع تعليمة `if` داخل تعليمة `if` أخرى. على سبيل المثال، فكر في برنامج مصرفي يحدد ما إذا كان عميل البنك مؤهلاً للحصول على سعر فائدة منخفض خاص على قرض. للتأهل، يجب أن يتوفر شرطان: (1) أن يكون العميل موظفًا حاليًا، و(2) أن يكون العميل قد تخرج حديثًا من الكلية (خلال السنتين الماضيتين). يوضح الشكل 7-4 مخطط انسيابي (Flowchart) لخوارزمية يمكن استخدامها في مثل هذا البرنامج.

Figure 4-7 Nested decision structures



Program 4-10 shows the code for the complete program.

Program 4-10

```
1 // This program demonstrates the nested if statement.
2 #include <iostream>
3 using namespace std;
4
5 int main()
6 {
7     char employed,    // Currently employed, Y or N
8         recentGrad;   // Recent graduate, Y or N
9
10    // Is the user employed and a recent graduate?
11    cout << "Answer the following questions\n";
12    cout << "with either Y for Yes or N for No.\n";
13    cout << "Are you employed? ";
14    cin >> employed;
15    cout << "Have you graduated from college ";
16    cout << "in the past two years? ";
17    cin >> recentGrad;
18
19    // Determine the user's loan qualifications.
20    if (employed == 'Y')
21    {
22        if (recentGrad == 'Y') //Nested if
23        {
24            cout << "You qualify for the special ";
25            cout << "interest rate.\n";
26        }
27    }
28    return 0;
29 }
30 }
```

Program Output with Example Input Shown in Bold

```
Answer the following questions
with either Y for Yes or N for No.
Are you employed? Y Enter
Have you graduated from college in the past two years? Y Enter
You qualify for the special interest rate.
```

Program Output with Different Example Input Shown in Bold

```
Answer the following questions
with either Y for Yes or N for No.
Are you employed? Y Enter
Have you graduated from college in the past two years? N Enter
```


لاحظ في التنفيذ الثاني للبرنامج 4-10 أن ناتج البرنامج لا يُعلم المستخدم ما إذا كان مؤهلاً للحصول على سعر الفائدة الخاص أم لا. إذا أدخل المستخدم الحرف 'N' (أو أي حرف آخر غير 'Y') للإجابة على سؤال "هل أنت موظف؟" أو "هل تخرجت حديثاً؟"، فإن البرنامج لا يقوم بإظهار أي رسالة توضح حالة التأهل. يمكن لتعليمة 'else' أن تعالج هذا الأمر، كما هو موضح في البرنامج 4-11.

Program 4-11

```
1 // This program demonstrates the nested if statement.
2 #include <iostream>
3 using namespace std;
4
5 int main()
6 {
7     char employed,    // Currently employed, Y or N
8         recentGrad;   // Recent graduate, Y or N
9
10    // Is the user employed and a recent graduate?
11    cout << "Answer the following questions\n";
12    cout << "with either Y for Yes or ";
13    cout << "N for No.\n";
14    cout << "Are you employed? ";
15    cin >> employed;
16    cout << "Have you graduated from college ";
17    cout << "in the past two years? ";
18    cin >> recentGrad;
19
20    // Determine the user's loan qualifications.
21    if (employed == 'Y')
22    {
23        if (recentGrad == 'Y') // Nested if
24        {
25            cout << "You qualify for the special ";
26            cout << "interest rate.\n";
27        }
28        else // Not a recent grad, but employed
29        {
30            cout << "You must have graduated from ";
31            cout << "college in the past two\n";
32            cout << "years to qualify.\n";
33        }
34    }
```



```

35     else // Not employed
36     {
37         cout << "You must be employed to qualify.\n";
38     }
39     return 0;
40 }

```

Program Output with Example Input Shown in Bold

Answer the following questions
with either Y for Yes or N for No.

Are you employed? **N**

Have you graduated from college in the past two years? **Y**

You must be employed to qualify.

Program 4-11 (continued)

Program Output with Different Example Input Shown in Bold

Answer the following questions
with either Y for Yes or N for No.

Are you employed? **Y**

Have you graduated from college in the past two years? **N**

You must have graduated from college in the past two years to qualify.

Program Output with Different Example Input Shown in Bold

Answer the following questions
with either Y for Yes or N for No.

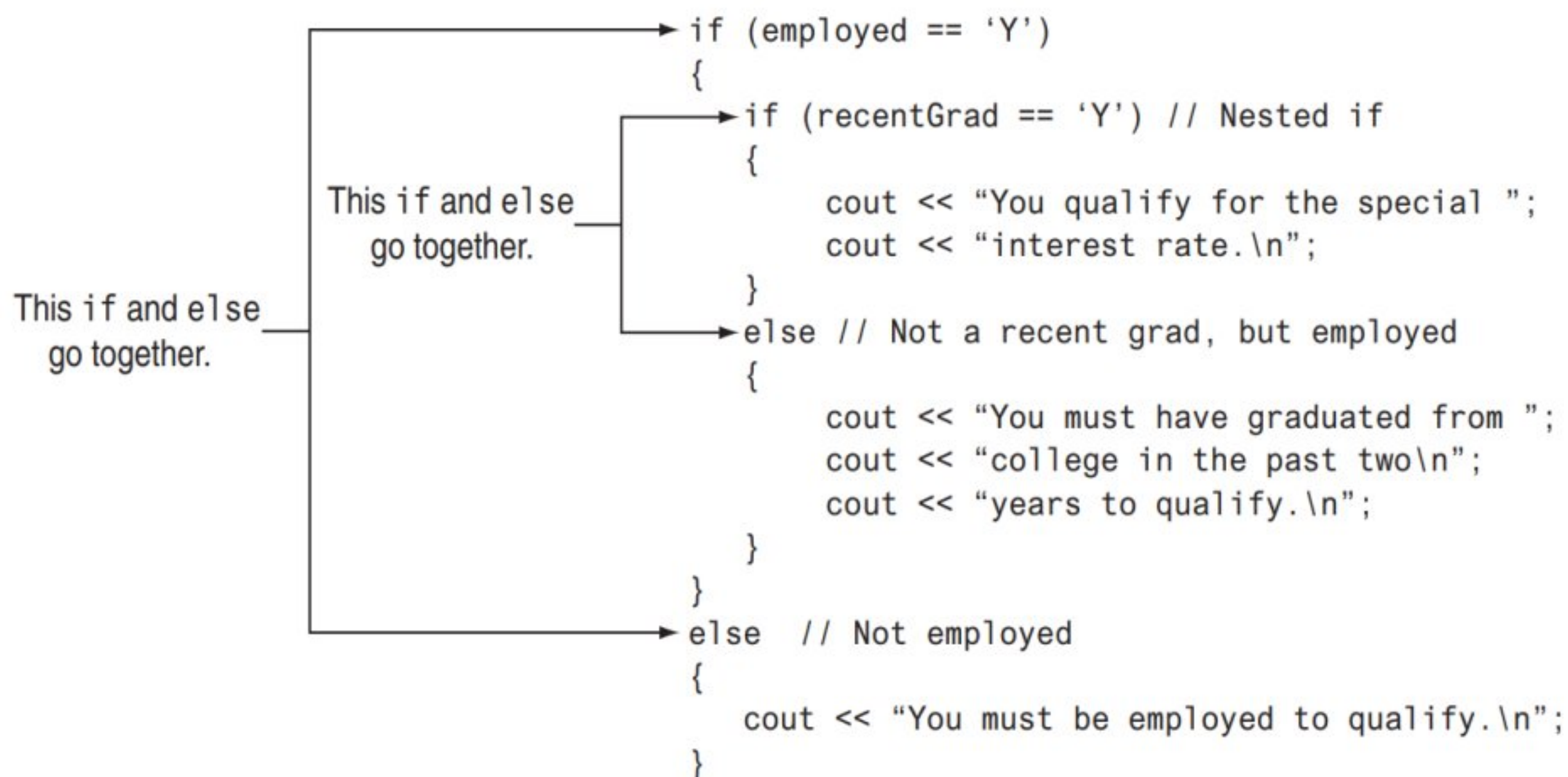
Are you employed? **Y**

Have you graduated from college in the past two years? **Y**

You qualify for the special interest rate.

- **Programming Style and Nested Decision Structures**

Figure 4-8 Proper indentation and alignment



- **Testing a Series of Conditions**

Dr. Suarez teaches a literature class and uses the following 10-point grading scale for all of his exams:

Test Score	Grade
90 and above	A
80–89	B
70–79	C
60–69	D
Below 60	F

He has asked you to write a program that will allow a student to enter a test score and then display the grade for that score. Here is the algorithm that you will use:

Ask the user to enter a test score.

Determine the grade in the following manner:

If the score is greater than or equal to 90, then the grade is A.

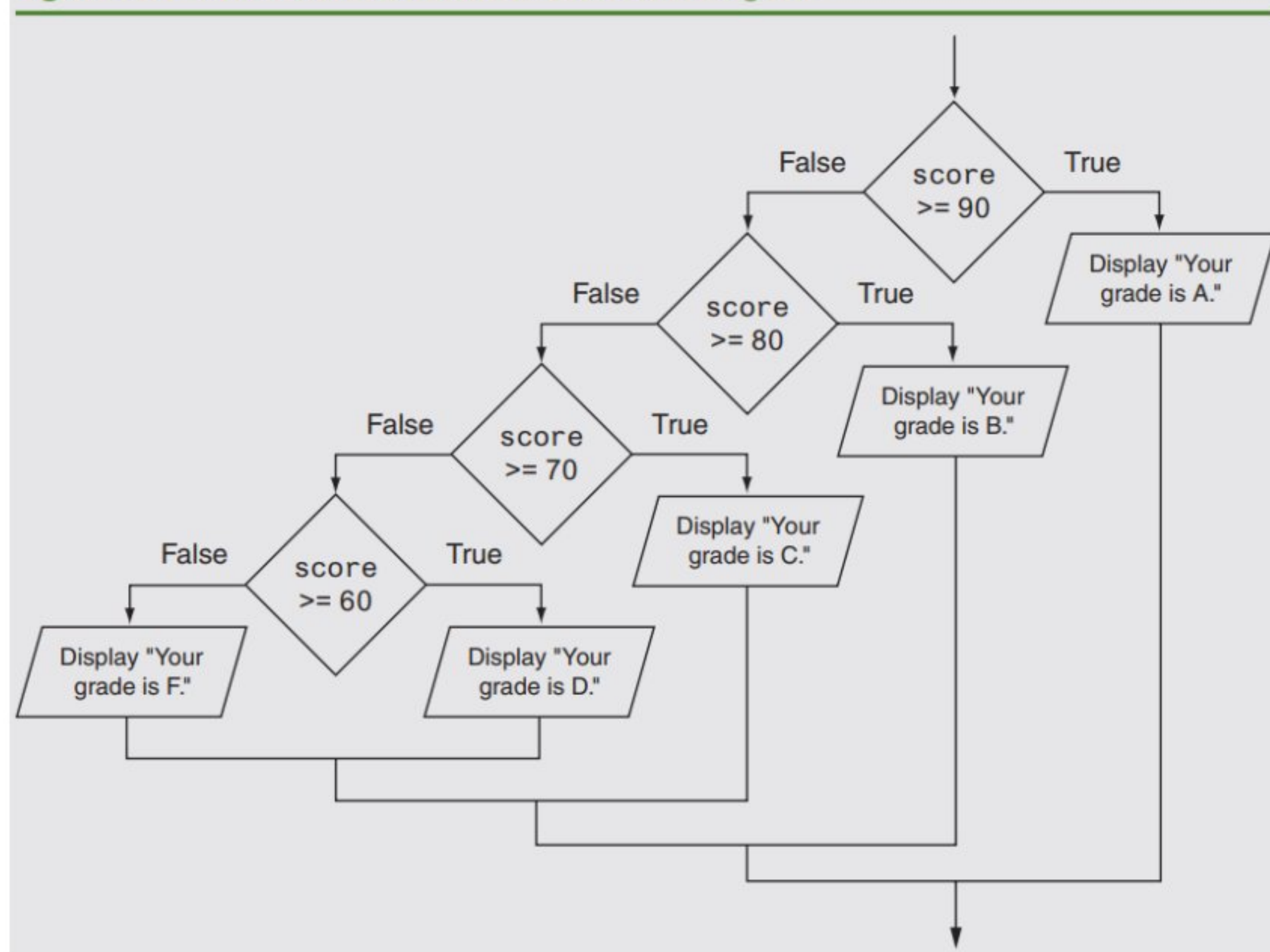
Otherwise, if the score is greater than or equal to 80, then the grade is B.

Otherwise, if the score is greater than or equal to 70, then the grade is C.

Otherwise, if the score is greater than or equal to 60, then the grade is D.

Otherwise, the grade is F.

Figure 4-9 Nested decision structure to determine a grade



Program 4-12

```
1 // This program uses nested if/else statements to assign a
2 // letter grade (A, B, C, D, or F) to a numeric test score.
3 #include <iostream>
4 using namespace std;
5
6 int main()
7 {
8     // Constants for grade thresholds
9     const int A_SCORE = 90,
10             B_SCORE = 80,
11             C_SCORE = 70,
12             D_SCORE = 60;
13
14     int testScore; // To hold a numeric test score
15
16     // Get the numeric test score.
17     cout << "Enter your numeric test score and I will\n";
18     cout << "tell you the letter grade you earned: ";
19     cin >> testScore;
20
21     // Determine the letter grade.
22     if (testScore >= A_SCORE)
23     {
24         cout << "Your grade is A.\n";
25     }
26     else
27     {
28         if (testScore >= B_SCORE)
29         {
30             cout << "Your grade is B.\n";
31         }
32         else
33         {
34             if (testScore >= C_SCORE)
35             {
36                 cout << "Your grade is C.\n";
37             }
38             else
39             {
40                 if (testScore >= D_SCORE)
41                 {
42                     cout << "Your grade is D.\n";
43                 }
44                 else
45                 {
46                     cout << "Your grade is F.\n";
47                 }
48             }
49         }
50     }
51
52     return 0;
53 }
```

Program Output with Example Input Shown in Bold

Enter your numeric test score and I will
tell you the letter grade you earned: **78**
Your grade is C.

Program Output with Different Example Input Shown in Bold

Enter your numeric test score and I will
tell you the letter grade you earned: **84**
Your grade is B.

4.6 The if/else if Statement

المفهوم: تقوم تعليمة `if/else if` باختبار سلسلة من الشروط. غالبًا ما يكون اختبار سلسلة من الشروط باستخدام تعليمة `if/else if` أسهل من استخدام مجموعة من تعليمات `if/else` المتداخلة.

إليك الشكل العام لتعليمة `if/else if`:

```
if (expression_1)
{
    statement
    statement
    etc.
}
else if (expression_2)
{
    statement
    statement
    etc.
}
Insert as many else if clauses as necessary
else
{
    statement
    statement
    etc.
}
```

If expression_1 is true these statements are executed, and the rest of the structure is ignored.

Otherwise, if expression_2 is true these statements are executed, and the rest of the structure is ignored.

These statements are executed if none of the expressions above are true.

تُعرف آخر عبارة `else`، والتي لا تحتوي على تعليمة `if` بعدها، باسم العبارة `else` الزائدة (Trailing Else). هذه العبارة اختيارية، ولكن في معظم الحالات ستستخدمها.



NOTE: The general format shows braces surrounding each block of conditionally executed statements. As with other forms of the `if` statement, the braces are required only when more than one statement is conditionally executed.

يوضح البرنامج 4-13 مثالاً على استخدام تعليمة `if/else if`. هذا البرنامج هو تعديل للبرنامج 4-12.

Program 4-13

```
1 // This program uses an if/else if statement to assign a
2 // letter grade (A, B, C, D, or F) to a numeric test score.
3 #include <iostream>
4 using namespace std;
5
6 int main()
7 {
8     // Constants for grade thresholds
9     const int A_SCORE = 90,
10             B_SCORE = 80,
11             C_SCORE = 70,
12             D_SCORE = 60;
13
14     int testScore; // To hold a numeric test score
15
16     // Get the numeric test score.
17     cout << "Enter your numeric test score and I will\n"
18          << "tell you the letter grade you earned: ";
19     cin >> testScore;
20
21     // Determine the letter grade.
22     if (testScore >= A_SCORE)
23         cout << "Your grade is A.\n";
24     else if (testScore >= B_SCORE)
25         cout << "Your grade is B.\n";
26     else if (testScore >= C_SCORE)
27         cout << "Your grade is C.\n";
28     else if (testScore >= D_SCORE)
29         cout << "Your grade is D.\n";
30     else
31         cout << "Your grade is F.\n";
32
```

(program continues)

Program 4-13 (continued)

```
33     return 0;
34 }
```

Program Output with Example Input Shown in Bold

Enter your numeric test score and I will
tell you the letter grade you earned: **78**
Your grade is C.

Program Output with Different Example Input Shown in Bold

Enter your numeric test score and I will
tell you the letter grade you earned: **84**
Your grade is B.

- **Using the Trailing else to Catch Errors**

يمكننا تعديل الكود كما هو موضح في البرنامج 4-14 بحيث تلتقط عبارة 'else' الزائدة أي درجة اختبار أقل من 0 وتعرض رسالة خطأ.

Program 4-14

```
1 // This program uses an if/else if statement to assign a
2 // letter grade (A, B, C, D, or F) to a numeric test score.
3 #include <iostream>
4 using namespace std;
5
6 int main()
7 {
8     // Constants for grade thresholds
9     const int A_SCORE = 90,
10             B_SCORE = 80,
11             C_SCORE = 70,
12             D_SCORE = 60;
13
14     int testScore; // To hold a numeric test score
15
16     // Get the numeric test score.
17     cout << "Enter your numeric test score and I will\n"
18           << "tell you the letter grade you earned: ";
19     cin >> testScore;
20
21     // Determine the letter grade.
22     if (testScore >= A_SCORE)
23         cout << "Your grade is A.\n";
24     else if (testScore >= B_SCORE)
25         cout << "Your grade is B.\n";
26     else if (testScore >= C_SCORE)
27         cout << "Your grade is C.\n";
28     else if (testScore >= D_SCORE)
29         cout << "Your grade is D.\n";
30     else if (testScore >= 0)
31         cout << "Your grade is F.\n";
32     else
33         cout << "Invalid test score.\n";
34
35     return 0;
36 }
```

Program Output with Example Input Shown in Bold

```
Enter your numeric test score and I will
tell you the letter grade you earned: -1 
Invalid test score.
```


• The if/else if Statement Compared to a Nested Decision Structure

ليس عليك استخدام تعليمة `if/else if` أبدًا لأنه يمكن تنفيذ منطقها باستخدام تعليمات `if/else` متداخلة. ومع ذلك، فإن سلسلة طويلة من تعليمات `if/else` المتداخلة لها عيوب معينة عند تصحيح الأخطاء في الكود:

- يمكن أن يصبح الكود معقدًا ويصعب فهمه.
- نظرًا لأن المسافات البادئة (Indentation) مهمة في التعليمات المتداخلة، يمكن أن تصبح سلسلة طويلة من تعليمات `if/else` المتداخلة طويلة جدًا بحيث لا يمكن عرضها على شاشة الكمبيوتر دون التمرير الأفقي. أيضًا، تميل التعليمات الطويلة إلى "الالتفاف" عند طباعتها على الورق، مما يجعل الكود أكثر صعوبة في القراءة.

منطق تعليمة `if/else if` عادةً ما يكون أسهل في المتابعة من سلسلة طويلة من تعليمات `if/else` المتداخلة. بالإضافة إلى ذلك، نظرًا لأن جميع العبارات تكون محاذاة في تعليمة `if/else if`، فإن طول الأسطر يميل إلى أن يكون أقصر.

4.7 The if Statement with Initialization

المفهوم: بدءًا من C++17، يمكن أن تحتوي تعليمة `if` بشكل اختياري على بند تهيئة (initialization clause).

إذا كنت تستخدم C++17 أو إصدارًا أحدث، يمكنك كتابة تعليمة `if` مع بند تهيئة اختياري يتم تنفيذه قبل تقييم التعبير الشرطي. إليك الشكل العام:

```
if (initialization; expression)
{
    statement;
    statement;
    // Place as many statements here as necessary.
}
```

لنلق نظرة على مثال. في الكود التالي، افترض أن `password` هو كائن من نوع `string`:

```
if (int len = password.length(); len < MIN_LENGTH)
{
    cout << "Your password is too short." << endl;
}
```


إليك الشكل العام لتعليمة `if/else` مع بند تهيئة:

```
if (initialization; expression)
{
    statement;
    statement;
    // Place as many statements here as necessary.
}
else
{
    statement;
    statement;
    // Place as many statements here as necessary.
}
```

عندما يتم تعريف متغير في بند التهيئة الخاص بتعليمة `if`، فإن نطاق هذا المتغير يقتصر على تعليمة `if` وكتلة `else` المرتبطة بها (إذا كانت موجودة). على سبيل المثال، الكود التالي يتسبب في حدوث خطأ:

```
if (int len = password.length(); len < MIN_LENGTH)
{
    cout << "Your password is too short." << endl;
}
// The following statement causes an error.
cout << "Your password is " << len << " characters long." << endl;
```

البيان الأخير لن يتم ترجمته لأنه يحاول استخدام المتغير `len`، الذي يمكن الوصول إليه فقط من خلال الكود الموجود داخل تعليمة `if`.
إليك مثال على تعليمة `if/else` تستخدم بند تهيئة:

```
if (int len = password.length(); len < MIN_LENGTH)
{
    cout << "Your password is too short." << endl;
}
else
{
    cout << "Your password has " << len << " characters." << endl;
}
```

في تعليمة `if/else if`، يمكنك تضمين بند تهيئة في أي من تعليمات `if`. إليك مثال:


```

if (int ulen = username.length(); ulen < MIN_LENGTH)
{
    cout << "Your username is too short." << endl;
}
else if (int plen = password.length(); plen < MIN_LENGTH)
{
    cout << "Your password is too short." << endl;
}
else
{
    cout << "Your username and password both"
    >> "meet the minimum length." << endl;
}

```

عندما يتم تعريف متغير في بند التهيئة الخاص بتعليمة `if/else if`، يبدأ نطاق هذا المتغير من نقطة تعريفه ويمتد حتى نهاية تعليمة `if/else if`.

4.8 Flags

المفهوم: العلم (Flag) هو متغير منطقي (Boolean) أو عددي (Integer) يشير إلى وجود شرط معين.

العلم (Flag) عادةً ما يكون متغيراً من النوع `bool` يشير إلى وجود شرط معين في البرنامج. عندما يتم تعيين قيمة المتغير إلى `false`، فهذا يعني أن الشرط غير موجود. وعندما يتم تعيينه إلى `true`، فهذا يعني أن الشرط موجود.

على سبيل المثال، لنفترض أن برنامجاً لحساب عمولات المبيعات يحتوي على متغير من النوع `bool`، يتم تعريفه وتهيئته كما يلي:

```
bool salesQuotaMet = false;
```

في البرنامج، يُستخدم المتغير `salesQuotaMet` كعلم للإشارة إلى ما إذا كان مندوب المبيعات قد حقق حصة المبيعات المطلوبة. عند تعريف المتغير، نقوم بتهيئته بالقيمة `false` لأننا لا نعرف بعد ما إذا كان مندوب المبيعات قد حقق الحصة. بافتراض وجود متغير يُسمى `sales` يحتوي على قيمة المبيعات، يمكن استخدام كود مشابه لما يلي لتعيين قيمة المتغير `salesQuotaMet`:

```
if (sales >= QUOTA_AMOUNT)
```



```

salesQuotaMet = true;
else
    salesQuotaMet = false;

```

نتيجة لهذا الكود، يمكن استخدام المتغير `salesQuotaMet` كعلم للإشارة إلى ما إذا تم تحقيق حصة المبيعات. لاحقاً في البرنامج، قد نختبر هذا العلم بالطريقة التالية:

```

if (salesQuotaMet)
    cout << "You have met your sales quota\n";

```

يعرض هذا الكود الرسالة "You have met your sales quota" إذا كان المتغير `salesQuotaMet` يحتوي على القيمة `true`. لاحظ أننا لم نستخدم العامل `==` لمقارنة المتغير `salesQuotaMet` بالقيمة `true` بشكل صريح. هذا الكود يعادل ما يلي:

```

if (salesQuotaMet == true)
    cout << "You have met your sales quota\n";

```

4.9 Logical Operators

المفهوم: العوامل المنطقية (Logical Operators) تربط تعبيرين أو أكثر من تعبيرات المقارنة في تعبير واحد، أو تعكس منطق التعبير.

Table 4-6 Logical Operators

Operator	Meaning	Effect
&&	AND	Connects two expressions into one. Both expressions must be true for the overall expression to be true.
	OR	Connects two expressions into one. One or both expressions must be true for the overall expression to be true. It is only necessary for one to be true, and it does not matter which.
!	NOT	The ! operator reverses the "truth" of an expression. It makes a true expression false, and a false expression true.

• The && Operator

العامل `&&` يُعرف باسم العامل المنطقي AND. يأخذ تعبيرين كمعاملات وينشئ تعبيراً يكون صحيحاً فقط عندما يكون كلا التعبيرين الفرعيين صحيحين. إليك مثال على تعليمة `if` تستخدم العامل `&&`:


```
if (temperature < 20 && minutes > 12)
    cout << "The temperature is in the danger zone";
```

في التعليمة أعلاه، يتم دمج تعبيرين مقارنة في تعبير واحد. سيتم تنفيذ تعليمة `cout` فقط إذا كانت قيمة `temperature` أقل من 20 وقيمة `minutes` أكبر من 12. إذا كان أي من الاختبارين المقارنة خاطئاً، فإن التعبير بأكمله يكون خاطئاً، ولن يتم تنفيذ تعليمة `cout`.



TIP: You must provide complete expressions on both sides of the && operator. For example, the following statement is not correct because the condition on the right side of the && operator is not a complete expression:

```
temperature > 0 && < 100
```

The expression must be rewritten as

```
temperature > 0 && temperature < 100
```

Table 4-7 Truth Table for the && Operator

Expression	Value of Expression
true && false	false (0)
false && true	false (0)
false && false	false (0)
true && true	true (1)

البرنامج 4-15 يؤدي عملية مشابهة للبرنامج 4-11، الذي يحدد ما إذا كان عميل البنك مؤهلاً للحصول على سعر فائدة خاص. هذا البرنامج يستخدم عاملاً منطقيًا.

Program 4-15

```
1 // This program demonstrates the && logical operator.
2 #include <iostream>
3 using namespace std;
4
5 int main()
6 {
7     char employed,    // Currently employed, Y or N
8         recentGrad;  // Recent graduate, Y or N
9
10    // Is the user employed and a recent graduate?
11    cout << "Answer the following questions\n";
12    cout << "with either Y for Yes or N for No.\n";
13
14    cout << "Are you employed? ";
15    cin >> employed;
16
17    cout << "Have you graduated from college "
18         << "in the past two years? ";
```


Program 4-15*(continued)*

```
19      cin >> recentGrad;
20
21      // Determine the user's loan qualifications.
22      if (employed == 'Y' && recentGrad == 'Y')
23      {
24          cout << "You qualify for the special "
25              << "interest rate.\n";
26      }
27      else
28      {
29          cout << "You must be employed and have\n"
30              << "graduated from college in the\n"
31              << "past two years to qualify.\n";
32      }
33      return 0;
34  }
```

Program Output with Example Input Shown in Bold

Answer the following questions
with either Y for Yes or N for No.

Are you employed? **Y**

Have you graduated from college in the past two years? **N**

You must be employed and have
graduated from college in the
past two years to qualify.

Program Output with Example Input Shown in Bold

Answer the following questions
with either Y for Yes or N for No.

Are you employed? **N**

Have you graduated from college in the past two years? **Y**

You must be employed and have
graduated from college in the
past two years to qualify.

Program Output with Example Input Shown in Bold

Answer the following questions
with either Y for Yes or N for No.

Are you employed? **Y**

Have you graduated from college in the past two years? **Y**

You qualify for the special interest rate.

• The || Operator

العامل '||' يُعرف باسم العامل المنطقي OR. يأخذ تعبيرين كمعاملات وينشئ تعبيرًا يكون صحيحًا عندما يكون أي من التعبيرين الفرعيين صحيحًا. إليك مثال على تعليمة 'if' تستخدم العامل '||':

```
if (temperature < 20 || temperature > 100)
    cout << "The temperature is in the danger zone";
```

سيتم تنفيذ تعليمة 'cout' إذا كانت قيمة 'temperature' أقل من 20 أو أكبر من 100. إذا كان أي من الاختبارين المقارنة صحيحًا، فإن التعبير بأكمله يكون صحيحًا، ويتم تنفيذ تعليمة 'cout'.



TIP: You must provide complete expressions on both sides of the || operator. For example, the following code is not correct because the condition on the right side of the || operator is not a complete expression:

```
temperature < 0 || > 100
```

The expression must be rewritten as

```
temperature < 0 || temperature > 100
```

Table 4-8 Truth Table for the || Operator

Expression	Value of the Expression
true false	true (1)
false true	true (1)
false false	false (0)
true true	true (1)

البرنامج 4-16 يقوم بإجراء اختبارات مختلفة لتحديد ما إذا كان الشخص مؤهلاً للحصول على قرض. هذا البرنامج يحدد ما إذا كان العميل يكسب على الأقل 35,000 دولار سنويًا، أو كان يعمل لأكثر من 5 سنوات.

Program 4-16

```
1 // This program demonstrates the logical || operator.
2 #include <iostream>
3 using namespace std;
4
5 int main()
6 {
7     // Constants for minimum income and years
8     const double MIN_INCOME = 35000.0;
9     const int MIN_YEARS = 5;
10
11     double income; // Annual income
12     int years;     // Years at the current job
13
14     // Get the annual income
15     cout << "What is your annual income? ";
16     cin >> income;
17
```



```

18 // Get the number of years at the current job.
19 cout << "How many years have you worked at "
20     << "your current job? ";
21 cin >> years;
22
23 // Determine the user's loan qualifications.
24 if (income >= MIN_INCOME || years > MIN_YEARS)
25     cout << "You qualify.\n";
26 else
27 {
28     cout << "You must earn at least $"
29         << MIN_INCOME << " or have been "
30         << "employed more than " << MIN_YEARS
31         << " years.\n";
32 }
33 return 0;
34 }

```

Program Output with Example Input Shown in Bold

What is your annual income? **40000**
 How many years have you worked at your current job? **2**
 You qualify.

Program Output with Example Input Shown in Bold

What is your annual income? **20000**
 How many years have you worked at your current job? **7**
 You qualify.

Program Output with Example Input Shown in Bold

What is your annual income? **30000**
 How many years have you worked at your current job? **3**
 You must earn at least \$35000 or have been employed more than 5 years.

• The ! Operator

العامل '!' يقوم بعملية NOT المنطقية. يأخذ معاملاً ويعكس قيمته المنطقية (صحيح أو خاطئ). بمعنى آخر، إذا كان التعبير صحيحاً، فإن العامل '!' يُرجع 'false'، وإذا كان التعبير خاطئاً، فإنه يُرجع 'true'. إليك مثال على تعليمة 'if' تستخدم العامل '!' :

```
if (!(temperature > 100))
```

```
    cout << "You are below the maximum temperature.\n";
```

أولاً، يتم اختبار التعبير '(temperature > 100)' لمعرفة ما إذا كان صحيحاً أو خاطئاً. ثم يتم تطبيق العامل '!' على هذه القيمة. إذا كان التعبير '(temperature > 100)' صحيحاً، فإن العامل '!' يُرجع 'false'. إذا كان خاطئاً، فإن العامل '!' يُرجع 'true'. في المثال، هذا يعادل السؤال: "هل درجة الحرارة ليست أكبر من 100؟"

Table 4-9 Truth Table for the ! Operator

Expression	Value of the Expression
!true	false (0)
!false	true (1)

البرنامج 4-17 يؤدي نفس المهمة التي يؤديها البرنامج 4-16. ومع ذلك، فإن تعليمة 'if' تستخدم العامل '!' لتحديد ما إذا كان المستخدم لا يكسب على الأقل 35,000 دولار، أو لم يكن في الوظيفة لأكثر من 5 سنوات.

Program 4-17

```
1 // This program demonstrates the logical ! operator.
2 #include <iostream>
3 using namespace std;
4
5 int main()
6 {
7     // Constants for minimum income and years
8     const double MIN_INCOME = 35000.0;
9     const int MIN_YEARS = 5;
10
11     double income; // Annual income
12     int years;     // Years at the current job
13
14     // Get the annual income
15     cout << "What is your annual income? ";
16     cin >> income;
17
18     // Get the number of years at the current job.
19     cout << "How many years have you worked at "
20          << "your current job? ";
```

Program 4-17 (continued)

```
21     cin >> years;
22
23     // Determine the user's loan qualifications.
24     if (!(income >= MIN_INCOME || years > MIN_YEARS))
25     {
26         cout << "You must earn at least $"
27              << MIN_INCOME << " or have been "
28              << "employed more than " << MIN_YEARS
29              << " years.\n";
30     }
31     else
32         cout << "You qualify.\n";
33     return 0;
34 }
```

The output of Program 4-17 is the same as Program 4-16.

• Precedence and Associativity of Logical Operators

Table 4-10 shows the precedence of C++'s logical operators, from highest to lowest.

Table 4-10 Precedence of Logical Operators

Logical Operators in Order of Precedence
!
&&

العامل '!' له أولوية أعلى من العديد من العوامل في لغة C++. لتجنب الأخطاء، يجب دائمًا وضع المعامل الخاص به بين قوسين إلا إذا كنت تتوي تطبيقه على متغير أو تعبير بسيط لا يحتوي على عوامل أخرى. على سبيل المثال، ضع في الاعتبار التعبيرات التالية:

!(x>2)
!x > 2

عوامل '&&' و '||' لها أولوية أقل من عوامل المقارنة (Relational Operators)، لذا فإن مشاكل الأولوية أقل احتمالاً للحدوث. ومع ذلك، إذا كنت غير متأكد، فلا بأس من استخدام الأقواس في جميع الأحوال.

(a > b) && (x < y) is the same as a > b && x < y
(x == y) || (b > a) is the same as x == y || b > a

العوامل المنطقية لها ارتباطية من اليسار إلى اليمين (left-to-right associativity). في التعبير التالي، يتم تقييم 'a < b' قبل 'y == z':

a < b || y == z
في التعبير التالي، يتم تقييم 'y == z' أولاً، لأن العامل '&&' له أولوية أعلى من العامل '||':

a < b || y == z && m > j

هذا التعبير يعادل:

(a < b) || ((y == z) && (m > j))

4.10 Checking Numeric Ranges with Logical Operators

المفهوم: العوامل المنطقية فعالة في تحديد ما إذا كان الرقم يقع ضمن نطاق معين أو خارجه.

عند تحديد ما إذا كان الرقم يقع ضمن نطاق رقمي معين، من الأفضل استخدام العامل '&&'. على سبيل المثال، تعليمة 'if' التالية تتحقق من قيمة 'x' لتحديد ما إذا كانت تقع ضمن النطاق من 20 إلى 40:

```
if (x >= 20 && x <= 40)  
cout << x << " is in the acceptable range.\n";
```

سيكون التعبير في تعليمة 'if' صحيحًا فقط عندما تكون قيمة 'x' أكبر من أو تساوي 20 وأقل من أو تساوي 40. يجب أن تكون 'x' ضمن النطاق من 20 إلى 40 حتى يكون هذا التعبير صحيحًا.

عند تحديد ما إذا كان الرقم خارج نطاق معين، من الأفضل استخدام العامل '||'. التعليمة التالية تحدد ما إذا كانت 'x' خارج النطاق من 20 إلى 40:


```
if (x < 20 || x > 40)
    cout << x << " is outside the acceptable range.\n";
```

من المهم عدم الخلط بين منطق هذه العوامل المنطقية. على سبيل المثال، تعليمة `if` التالية لن تكون صحيحة أبدًا:

```
if (x < 20 && x > 40)
    cout << x << " is outside the acceptable range.\n";
```

من الواضح أن `x` لا يمكن أن تكون أقل من 20 وفي نفس الوقت أكبر من 40.



NOTE: C++ does not allow you to check numeric ranges with expressions such as `5 < x < 20`. Instead, you must use a logical operator to connect two relational expressions, as previously discussed.

4.11 Menus

المفهوم: يمكنك استخدام تعليمات `if/else` متداخلة أو تعليمة `if/else if` لإنشاء برامج تعتمد على القوائم (Menu-Driven Programs). البرنامج الذي يعتمد على القوائم يسمح للمستخدم بتحديد الإجراء المطلوب من خلال اختياره من قائمة بالإجراءات.

القائمة هي شاشة تعرض مجموعة من الخيارات يمكن للمستخدم الاختيار من بينها. على سبيل المثال، قد يعرض برنامج يدير قائمة بريدية القائمة التالية:

1. إضافة اسم إلى القائمة.
2. إزالة اسم من القائمة.
3. تغيير اسم في القائمة.
4. طباعة القائمة.
5. إنهاء البرنامج.

يختار المستخدم أحد العمليات عن طريق إدخال رقمها. على سبيل المثال، يؤدي إدخال الرقم 4 إلى طباعة القائمة البريدية، بينما يؤدي إدخال الرقم 5 إلى إنهاء البرنامج. يمكن استخدام تعليمات `if/else` متداخلة أو هيكل `if/else if` لإعداد مثل هذه القائمة. بعد إدخال المستخدم لرقم، يقارن البرنامج الرقم بالخيارات المتاحة وينفذ التعليمات التي تقوم بتنفيذ العملية المحددة.

البرنامج 4-18 يحسب تكاليف العضوية في نادي صحي. يحتوي النادي على ثلاث باقات عضوية للاختيار من بينها: عضوية البالغين القياسية، عضوية الأطفال، وعضوية كبار السن. يعرض البرنامج قائمة تسمح للمستخدم باختيار الباقة المطلوبة ثم يحسب تكلفة العضوية.

Program 4-18

```
1 // This program displays a menu and asks the user to make a
2 // selection. An if/else if statement determines which item
3 // the user has chosen.
4 #include <iostream>
5 #include <iomanip>
6 using namespace std;
7
8 int main()
9 {
10     int choice;    // To hold a menu choice
11     int months;    // To hold the number of months
12     double charges; // To hold the monthly charges
13
14     // Constants for membership rates
15     const double ADULT = 40.0,
16                 SENIOR = 30.0,
17                 CHILD = 20.0;
18
19     // Constants for menu choices
20     const int ADULT_CHOICE = 1,
21             CHILD_CHOICE = 2,
22             SENIOR_CHOICE = 3,
23             QUIT_CHOICE = 4;
24
25     // Display the menu and get a choice.
26     cout << "\t\tHealth Club Membership Menu\n\n"
27          << "1. Standard Adult Membership\n"
28          << "2. Child Membership\n"
29          << "3. Senior Citizen Membership\n"
30          << "4. Quit the Program\n\n"
31          << "Enter your choice: ";
32     cin >> choice;
33
34     // Set the numeric output formatting.
35     cout << fixed << showpoint << setprecision(2);
36
37     // Respond to the user's menu selection.
38     if (choice == ADULT_CHOICE)
39     {
40         cout << "For how many months? ";
41         cin >> months;
42         charges = months * ADULT;
43         cout << "The total charges are $" << charges << endl;
44     }
45     else if (choice == CHILD_CHOICE)
46     {
47         cout << "For how many months? ";
```


Program 4-18*(continued)*

```
48         cin >> months;
49         charges = months * CHILD;
50         cout << "The total charges are $" << charges << endl;
51     }
52     else if (choice == SENIOR_CHOICE)
53     {
54         cout << "For how many months? ";
55         cin >> months;
56         charges = months * SENIOR;
57         cout << "The total charges are $" << charges << endl;
58     }
59     else if (choice == QUIT_CHOICE)
60     {
61         cout << "Program ending.\n";
62     }
63     else
64     {
65         cout << "The valid choices are 1 through 4. Run the\n"
66             << "program again and select one of those.\n";
67     }
68     return 0;
69 }
```

Program Output with Example Input Shown in Bold**Health Club Membership Menu**

1. Standard Adult Membership
2. Child Membership
3. Senior Citizen Membership
4. Quit the Program

Enter your choice: **3**
For how many months? **6**
The total charges are \$180.00

لنلق نظرة أقرب على البرنامج:

الأسطر 10-12 تُعرف المتغيرات التالية:

- المتغير `choice` سيحتفظ باختيار المستخدم من القائمة.
- المتغير `months` سيحتفظ بعدد أشهر العضوية في النادي الصحي.
- المتغير `charges` سيحتفظ بإجمالي التكاليف.

الأسطر 15-17 تُعرف ثوابت شهرية لأسعار العضوية للبالغين، وكبار السن، والأطفال.

الأسطر 20-23 تُعرف ثوابت لخيارات القائمة.

- الأسطر 26-32 تعرض القائمة وتحصل على اختيار المستخدم.
- السطر 35 يضبط تنسيق الإخراج الرقمي للأعداد العشرية.
- الأسطر 38-67 هي تعليمة `if/else if` تحدد اختيار المستخدم من القائمة بالطريقة التالية:
- إذا اختار المستخدم 1 من القائمة (عضوية البالغين)، يتم تنفيذ التعليمات في الأسطر 40-43.
 - وإلا، إذا اختار المستخدم 2 من القائمة (عضوية الأطفال)، يتم تنفيذ التعليمات في الأسطر 47-50.
 - وإلا، إذا اختار المستخدم 3 من القائمة (عضوية كبار السن)، يتم تنفيذ التعليمات في الأسطر 54-57.
 - وإلا، إذا اختار المستخدم 4 من القائمة (إنهاء البرنامج)، يتم تنفيذ التعليمة في السطر 61.
 - إذا أدخل المستخدم أي خيار غير 1، 2، 3، أو 4، يتم تنفيذ بند `else` في الأسطر 63-67، والذي يعرض رسالة خطأ.

4.12 Focus on Software Engineering: Validating User Input

المفهوم: طالما يدخل المستخدم بيانات غير صحيحة في البرنامج، فإن البرنامج سيُنتج نتائج غير صحيحة. يجب أن تكون البرامج مكتوبة لتصفية المدخلات غير الصحيحة.

ربما تكون المقولة الأكثر شهرة في عالم الكمبيوتر هي "إذا أدخلت قمامة، ستخرج قمامة" (Garbage in, garbage out). جودة ناتج البرنامج تعتمد على جودة المدخلات، لذا يجب أن تحرص على ألا تدخل بيانات غير صحيحة إلى برامجك. التحقق من صحة المدخلات (Input Validation) هو عملية فحص البيانات التي يدخلها المستخدم إلى البرنامج وتحديد ما إذا كانت صالحة. يجب أن يقدم البرنامج الجيد تعليمات واضحة حول نوع المدخلات المقبولة، وألا يفترض أن المستخدم اتبع هذه التعليمات. إليك بعض الأمثلة على عمليات التحقق من صحة المدخلات التي تقوم بها البرامج:

- التحقق من أن الأرقام تقع ضمن نطاق القيم الممكنة: على سبيل المثال، هناك 168 ساعة في الأسبوع. من المستحيل أن يكون الشخص في العمل لأكثر من 168 ساعة في أسبوع واحد.
- التحقق من "معقولية" القيم: على الرغم من أنه قد يكون من الممكن أن يعمل الشخص لمدة 168 ساعة في الأسبوع، إلا أن هذا غير محتمل.
- التحقق من العناصر المختارة من القوائم أو الخيارات المتاحة: للتأكد من أنها خيارات متاحة.
- التحقق من القيم التي قد تسبب مشاكل: مثل القسمة على صفر.
- البرنامج 4-19 هو برنامج لتسجيل درجات الاختبارات يرفض أي درجة أقل من 0 أو أكبر من 100.

Program 4-19

```
1 // This test scoring program does not accept test scores
2 // that are less than 0 or greater than 100.
3 #include <iostream>
4 using namespace std;
5
6 int main()
7 {
8     // Constants for grade thresholds
```


Program 4-19 (continued)

```
9      const int A_SCORE = 90,
10             B_SCORE = 80,
11             C_SCORE = 70,
12             D_SCORE = 60,
13             MIN_SCORE = 0,      // Minimum valid score
14             MAX_SCORE = 100;    // Maximum valid score
15
16      int testScore; // To hold a numeric test score
17
18      // Get the numeric test score.
19      cout << "Enter your numeric test score and I will\n"
20            << "tell you the letter grade you earned: ";
21      cin >> testScore;
22
23      // Validate the input and determine the grade.
24      if (testScore >= MIN_SCORE && testScore <= MAX_SCORE)
25      {
26          // Determine the letter grade.
27          if (testScore >= A_SCORE)
28              cout << "Your grade is A.\n";
29          else if (testScore >= B_SCORE)
30              cout << "Your grade is B.\n";
31          else if (testScore >= C_SCORE)
32              cout << "Your grade is C.\n";
33          else if (testScore >= D_SCORE)
34              cout << "Your grade is D.\n";
35          else
36              cout << "Your grade is F.\n";
37      }
38      else
39      {
40          // An invalid score was entered.
41          cout << "That is an invalid score. Run the program\n"
42                << "again and enter a value in the range of\n"
43                << MIN_SCORE << " through " << MAX_SCORE << ".\n";
44      }
45
46      return 0;
47  }
```

Program Output with Example Input Shown in Bold

Enter your numeric test score and I will
tell you the letter grade you earned: **-1**
That is an invalid score. Run the program
again and enter a value in the range of
0 through 100.

Program Output with Example Input Shown in Bold

Enter your numeric test score and I will
tell you the letter grade you earned: **81**
Your grade is B.

4.13 Comparing Characters and Strings

المفهوم: يمكن أيضاً استخدام عوامل المقارنة (Relational Operators) لمقارنة الأحرف وكائنات النصوص (string objects).

في وقت سابق من هذا الفصل، تعلمت استخدام عوامل المقارنة لمقارنة القيم الرقمية. يمكن أيضاً استخدامها لمقارنة الأحرف وكائنات النصوص.

• Comparing Characters

يتم تخزين الأحرف في الذاكرة كأعداد صحيحة. في معظم الأنظمة، هذا العدد الصحيح يمثل قيمة ASCII للحرف. على سبيل المثال، الحرف 'A' يمثل الرقم 65، والحرف 'B' يمثل الرقم 66، وهكذا. يوضح الجدول 4-11 الأرقام ASCII التي تتوافق مع بعض الأحرف الشائعة الاستخدام.

Table 4-11 ASCII Values of Commonly Used Characters

Character	ASCII Value
'0'–'9'	48–57
'A'–'Z'	65–90
'a'–'z'	97–122
Blank	32
Period	46

عند مقارنة حرفين، يتم في الواقع مقارنة قيم ASCII الخاصة بهما. $'A' < 'B'$ لأن قيمة ASCII للحرف 'A' (65) أقل من قيمة ASCII للحرف 'B' (66). وبالمثل، $'2' > '1'$ لأن قيمة ASCII للحرف '1' (49) أقل من قيمة ASCII للحرف '2' (50). ومع ذلك، كما هو موضح في الجدول 4-11، الأحرف الصغيرة (lowercase) لها قيم ASCII أعلى من الأحرف الكبيرة (uppercase)، لذا $'a' > 'Z'$. يوضح البرنامج 4-20 كيفية مقارنة الأحرف باستخدام عوامل المقارنة.

Program 4-20

```
1 // This program demonstrates how characters can be
2 // compared with the relational operators.
3 #include <iostream>
4 using namespace std;
5
6 int main()
7 {
8     char ch;
9 }
```


Program 4-20 (continued)

```

10      // Get a character from the user.
11      cout << "Enter a digit or a letter: ";
12      ch = cin.get();
13
14      // Determine what the user entered.
15      if (ch >= '0' && ch <= '9')
16          cout << "You entered a digit.\n";
17      else if (ch >= 'A' && ch <= 'Z')
18          cout << "You entered an uppercase letter.\n";
19      else if (ch >= 'a' && ch <= 'z')
20          cout << "You entered a lowercase letter.\n";
21      else
22          cout << "That is not a digit or a letter.\n";
23
24      return 0;
25  }

```

Program Output with Example Input Shown in Bold

Enter a digit or a letter: **t**
 You entered a lowercase letter.

Program Output with Example Input Shown in Bold

Enter a digit or a letter: **v**
 You entered an uppercase letter.

Program Output with Example Input Shown in Bold

Enter a digit or a letter: **5**
 You entered a digit.

Program Output with Example Input Shown in Bold

Enter a digit or a letter: **&**
 That is not a digit or a letter.

- **Comparing string Objects**

يمكن أيضًا مقارنة كائنات النصوص (string objects) باستخدام عوامل المقارنة. كما هو الحال مع الأحرف الفردية، عند مقارنة كائني نصوص، يتم في الواقع مقارنة قيم ASCII للأحرف التي تتكون منها النصوص. على سبيل المثال، افترض أن التعريفات التالية موجودة في برنامج:

```

string str1 = "ABC";
string str2 = "XYZ";

```

يعتبر كائن النص `str1` أقل من كائن النص `str2` لأن الأحرف "ABC" تسبق أبجديًا (لها قيم ASCII أقل من) الأحرف "XYZ". لذلك، ستؤدي تعليمة `if` التالية إلى عرض الرسالة "str1 is less than str2" على الشاشة:

if (str1 < str2)

cout << "str1 is less than str2";

يتم مقارنة كل حرف في المعامل الأول مع الحرف في الموضع المقابل في المعامل الثاني واحدًا تلو الآخر. إذا تطابقت جميع الأحرف في كائني النصوص، فإن النصين متساويان. يمكن تحديد العلاقات الأخرى إذا لم تتطابق الأحرف في المواضع المقابلة. يكون المعامل الأول أقل من المعامل الثاني إذا كان الحرف الأول غير المتطابق في المعامل الأول أقل من نظيره في المعامل الثاني. وبالمثل، يكون المعامل الأول أكبر من المعامل الثاني إذا كان الحرف الأول غير المتطابق في المعامل الأول أكبر من نظيره في المعامل الثاني.

على سبيل المثال، افترض أن البرنامج يحتوي على التعريفات التالية:

string name1 = "Mary";

string name2 = "Mark";

القيمة في `name1`، وهي "Mary"، أكبر من القيمة في `name2`، وهي "Mark". هذا لأن الأحرف الثلاثة الأولى في `name1` لها نفس قيم ASCII للأحرف الثلاثة الأولى في `name2`، ولكن الحرف 'y' في الموضع الرابع من "Mary" له قيمة ASCII أكبر من الحرف 'k' في الموضع المقابل من "Mark".

يمكن استخدام أي من عوامل المقارنة لمقارنة كائني نصوص. إليك بعض المقارنات الصحيحة بين `name1` و `name2`:

name1 > name2 // true

name1 <= name2 // false

name1 != name2 // true

يمكن أيضًا مقارنة كائنات النصوص مع النصوص الحرفية (string literals):

name1 < "Mary Jane" // true

يوضح البرنامج 4-21 بشكل أكبر كيفية استخدام عوامل المقارنة مع كائنات النصوص.

Program 4-21

```
1 // This program uses relational operators to compare a string
2 // entered by the user with valid part numbers.
3 #include <iostream>
4 #include <iomanip>
5 #include <string>
6 using namespace std;
7
8 int main()
9 {
10     const double PRICE_A = 249.0, // Price for part A
11         PRICE_B = 199.0; // Price for part B
12
13     string partNum; // Holds a part number
14
15     // Display available parts and get the user's selection
16     cout << "The headphone part numbers are:\n"
17         << "Noise canceling: part number S-29A \n"
18         << "Wireless: part number S-29B \n"
19         << "Enter the part number of the headphones you\n"
```

Program 4-21 (continued)

```
20         << "wish to purchase: ";
21     cin >> partNum;
22
23     // Set the numeric output formatting
24     cout << fixed << showpoint << setprecision(2);
25
26     // Determine and display the correct price
27     if (partNum == "S-29A")
28         cout << "The price is $" << PRICE_A << endl;
29     else if (partNum == "S-29B")
30         cout << "The price is $" << PRICE_B << endl;
31     else
32         cout << partNum << " is not a valid part number.\n";
33     return 0;
34 }
```

Program Output with Example Input Shown in Bold

```
The headphone part numbers are:
Noise canceling: part number S-29A
Wireless: part number S-29B
Enter the part number of the headphones you
wish to purchase: S-29A 
The price is $249.00
```


4.14 The Conditional Operator

المفهوم: يمكنك استخدام العامل الشرطي (Conditional Operator) لإنشاء تعبيرات قصيرة تعمل مثل تعليمات `if/else`.

العامل الشرطي هو عامل قوي وفريد. يوفر طريقة مختصرة للتعبير عن تعليمة `if/else` بسيطة. يتكون العامل من علامة الاستفهام (`?`) والنقطتين (`:`). وشكله العام هو:

```
expression ? expression : expression;
```

إليك مثال على تعليمة تستخدم العامل الشرطي:

```
x < 0 ? y = 10 : z = 20;
```

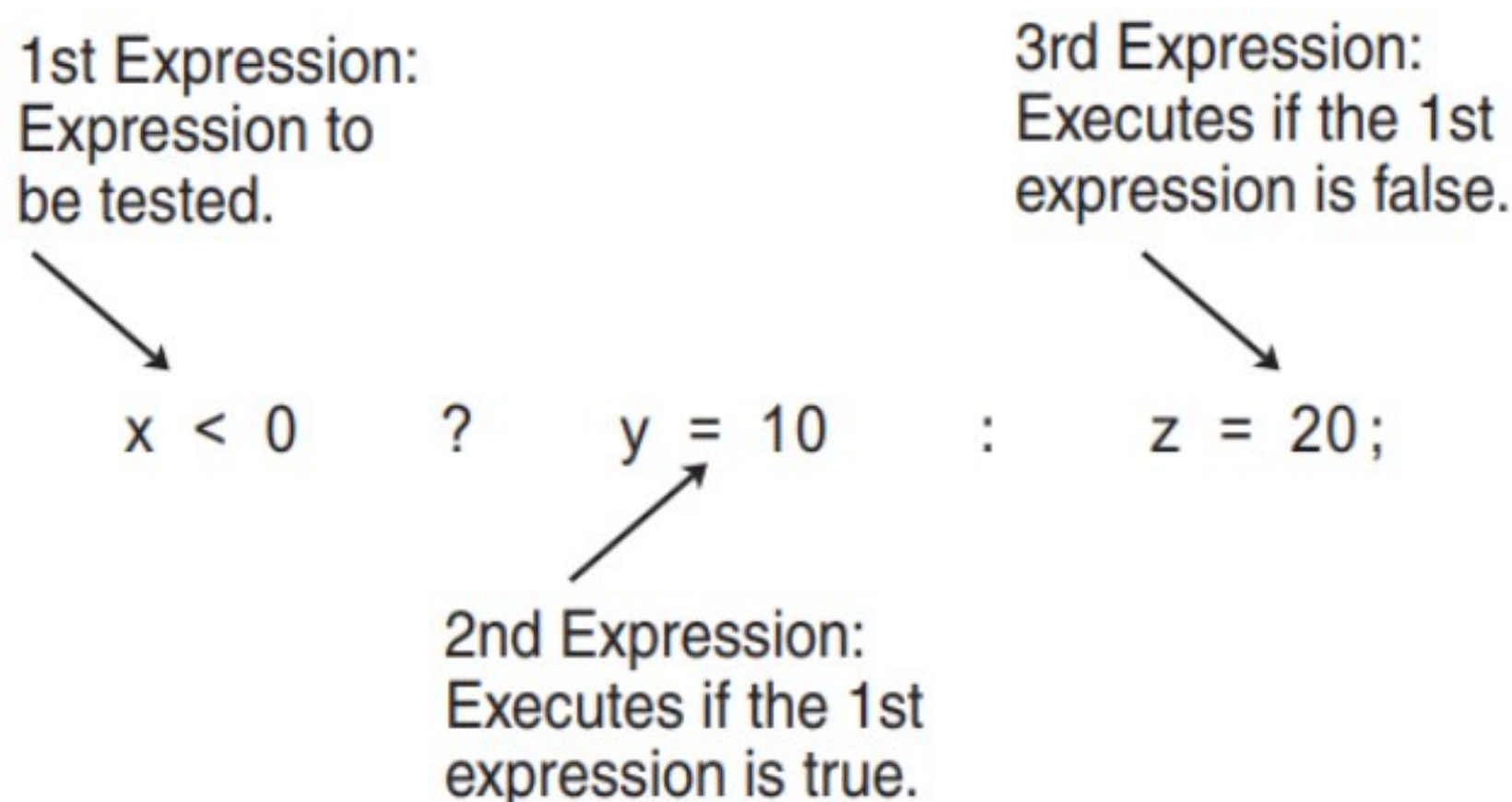
التعليمة أعلاه تسمى تعبير شرطي (Conditional Expression) وتتكون من ثلاثة تعبيرات فرعية مفصولة بعلامة الاستفهام (`?`) والنقطتين (`:`). التعابير هي `x < 0`، `y = 10`، و `z = 20`، كما هو موضح هنا:

```
x < 0 ? y = 10 : z = 20;    ⇔    if (x < 0)
                                y = 10;
                                else
                                z = 20;
```



NOTE: Since it takes three operands, the conditional operator is considered a *ternary* operator.

Figure 4-10 Conditional expression



If it helps, you can put parentheses around the subexpressions, as in the following:

```
(x < 0) ? (y = 10) : (z = 20);
```


Using the Value of a Conditional Expression

تذكر، في لغة C++ جميع التعبيرات لها قيمة، وهذا يشمل التعبير الشرطي. إذا كان التعبير الفرعي الأول صحيحًا، فإن قيمة التعبير الشرطي هي قيمة التعبير الفرعي الثاني. وإلا، فإنها تكون قيمة التعبير الفرعي الثالث. إليك مثال على عبارة تعيين باستخدام قيمة التعبير الشرطي:

a = x > 100 ? 0 : 1;

القيمة التي يتم تعيينها لـ 'a' ستكون إما '0' أو '1'، اعتمادًا على ما إذا كانت 'x' أكبر من '100'. يمكن التعبير عن هذه العبارة باستخدام عبارة 'if/else' كما يلي:

if (x > 100)

a = 0;

else

a = 1;

يمكن استخدام البرنامج 4-22 لمساعدة مستشارة في حساب رسومها. معدلها هو 50.00 دولار في الساعة، ولكن الحد الأدنى للرسوم هو 5 ساعات. يتم استخدام العامل الشرطي في عبارة تضمن ألا يقل عدد الساعات عن خمس ساعات.

Program 4-22

```
1 // This program calculates a consultant's charges at $50
2 // per hour, for a minimum of 5 hours. The ?: operator
3 // adjusts hours to 5 if less than 5 hours were worked.
4 #include <iostream>
5 #include <iomanip>
6 using namespace std;
7
8 int main()
9 {
10     const double PAY_RATE = 50.0; // Hourly pay rate
11     const int MIN_HOURS = 5;      // Minimum billable hours
12     double hours,                 // Hours worked
13           charges;                // Total charges
14
15     // Get the hours worked.
16     cout << "How many hours were worked? ";
17     cin >> hours;
18
19     // Determine the hours to charge for.
20     hours = hours < MIN_HOURS ? MIN_HOURS : hours;
21
22     // Calculate and display the charges.
23     charges = PAY_RATE * hours;
24     cout << fixed << showpoint << setprecision(2)
25           << "The charges are $" << charges << endl;
26     return 0;
27 }
```

Program Output with Example Input Shown in Bold

How many hours were worked? **10**
The charges are \$500.00

Program Output with Example Input Shown in Bold

How many hours were worked? **2**
The charges are \$250.00

لاحظ في السطر 11 أنه تم تعريف ثابت باسم `MIN\_HOURS` لتمثيل الحد الأدنى لعدد الساعات، وهو 5. إليك العبارة في السطر 20، مع التعبير الشرطي:

```
hours = hours < MIN_HOURS ? MIN_HOURS : hours;
```

إذا كانت القيمة في المتغير `hours` أقل من 5، فإن القيمة 5 تُخزن في `hours`. وإلا، فإن `hours` تحتفظ بقيمتها الحالية. وبالتالي، لن يكون للمتغير `hours` قيمة أقل من 5 عند استخدامه في العبارة التالية، التي تحسب رسوم المستشارة.

كما ترى، فإن العامل الشرطي يمنحك القدرة على تضمين قوة اتخاذ القرار في سطر برمجي موجز. وبقليل من الخيال، يمكن تطبيقه على العديد من المشكلات البرمجية الأخرى. على سبيل المثال، ضع في اعتبارك العبارة التالية:

```
cout << "Your grade is: " << (score < 60 ? "Fail." : "Pass.");
```

إذا كنت ستستخدم عبارة `if/else`، فسيتم كتابة العبارة أعلاه كما يلي:

```
if (score < 60)
```

```
    cout << "Your grade is: Fail.";
```

```
else
```

```
    cout << "Your grade is: Pass.";
```



NOTE: The parentheses are placed around the conditional expression because the << operator has higher precedence than the ?: operator. Without the parentheses, just the value of the expression `score < 60` would be sent to `cout`.

4.15 The switch Statement

المفهوم: عبارة `switch` تتيح لقيمة متغير أو تعبير أن تحدد نقطة الانتقال (التفرع) في البرنامج.

يحدث التفرع عندما يؤدي جزء من البرنامج إلى تنفيذ جزء آخر. تسمح عبارة `if/else if` لبرنامجك بالانتقال إلى أحد المسارات المحتملة العديدة. حيث تقوم بإجراء سلسلة من الاختبارات (عادةً ما تكون علائقية) وتنتقل إلى مسار معين عندما يكون أحد هذه الاختبارات صحيحًا. تعتبر عبارة `switch` آلية مشابهة، ولكنها تختبر قيمة تعبير صحيح (integer) ثم تستخدم هذه القيمة لتحديد مجموعة العبارات التي سيتم الانتقال إليها. إليك الشكل العام لعبارة `switch`:

```
switch (IntegerExpression)
{
    case ConstantExpression:
        // place one or more
        // statements here

    case ConstantExpression:
        // place one or more
        // statements here

    // case statements may be repeated as many
    // times as necessary

    default:
        // place one or more
        // statements here
}
```


يبدأ السطر الأول من العبارة بالكلمة 'switch'، يليها تعبير صحيح داخل الأقواس. يمكن أن يكون هذا التعبير واحدًا مما يلي:

- متغير من أي نوع بيانات صحيح (بما في ذلك 'char').
- تعبير قيمته من أي نوع بيانات صحيح.

في السطر التالي، يبدأ كتلة تحتوي على عدة عبارات 'case'. يتم تنسيق كل عبارة 'case' بالشكل التالي:

case ConstantExpression:

// place one or more

// statements here

بعد الكلمة 'case' يأتي تعبير ثابت (يجب أن يكون من نوع بيانات صحيح)، يليه نقطتان رأسيتان (':'). يمكن أن يكون التعبير الثابت قيمة عددية صحيحة مباشرة ('integer literal') أو ثابتًا معرفًا مسبقًا من نوع صحيح.

عبارة 'case' تشير إلى بداية قسم من العبارات. ينتقل البرنامج إلى هذه العبارات إذا كانت قيمة التعبير الموجود في 'switch' تطابق قيمة التعبير الموجود في 'case'.



WARNING! The expression of each case statement in the block must be unique.



NOTE: The expression following the word case must be an integer literal or constant. It cannot be a variable, and it cannot be an expression such as `x < 22` or `n == 50`.

يأتي قسم 'default' اختياريًا بعد جميع عبارات 'case'. ينتقل البرنامج إلى هذا القسم إذا لم يتطابق أي من تعبيرات 'case' مع تعبير 'switch'. وبالتالي، يعمل هذا القسم بشكل مشابه لـ 'else' الأخير في عبارة 'if/else if'.

يوضح البرنامج 4-23 كيفية عمل عبارة 'switch' البسيطة.

Program 4-23

```
1 // The switch statement in this program tells the user something
2 // he or she already knows: the data just entered!
3 #include <iostream>
4 using namespace std;
5
6 int main()
7 {
8     char choice;
9
10    cout << "Enter A, B, or C: ";
11    cin >> choice;
```



```

12     switch (choice)
13     {
14         case 'A': cout << "You entered A.\n";
15                     break;
16         case 'B': cout << "You entered B.\n";
17                     break;
18         case 'C': cout << "You entered C.\n";
19                     break;
20         default: cout << "You did not enter A, B, or C!\n";
21     }
22     return 0;
23 }

```

(program continues)

Program 4-23 (continued)

Program Output with Example Input Shown in Bold

Enter A, B, or C: **B**
 You entered B.

Program Output with Example Input Shown in Bold

Enter A, B, or C: **F**
 You did not enter A, B, or C!

عبارة 'case' الأولى هي 'A': case 'B': والثانية هي 'C': case 'C': هي الثالثة هي 'A' هذه العبارات تحدد النقاط التي سيذهب إليها البرنامج إذا كان المتغير 'choice' يحتوي على القيم 'A' أو 'B' أو 'C'. (تذكر أن المتغيرات والحروف النصية تُعتبر قيمًا صحيحة.) يتم الانتقال إلى قسم 'default' إذا أدخل المستخدم أي شيء غير 'A' أو 'B' أو 'C'. Notice the break statements in the case 'A', case 'B', and case 'C' sections.

```

switch (choice)
{
    case 'A':cout << "You entered A.\n";
                break;
    case 'B':cout << "You entered B.\n";
                break;
    case 'C':cout << "You entered C.\n";
                break;
    default: cout << "You did not enter A, B, or C!\n";
}

```

عبارات 'case' تُظهر للبرنامج من أين يبدأ التنفيذ داخل الكتلة، وعبارات 'break' تُظهر له أين يتوقف. بدون وجود عبارات 'break'، سيستمر البرنامج في تنفيذ جميع الأسطر من عبارة 'case' المطابقة حتى نهاية الكتلة.



NOTE: The default section (or the last case section, if there is no default) does not need a break statement. Some programmers prefer to put one there anyway, for consistency.

Program 4-24 is a modification of Program 4-23, without the break statements.

Program 4-24

```
1 // The switch statement in this program tells the user something
2 // he or she already knows: the data just entered!
3 #include <iostream>
4 using namespace std;
5
6 int main()
7 {
8     char choice;
9
10    cout << "Enter A, B, or C: ";
11    cin >> choice;
12    // The following switch is
13    // missing its break statements!
14    switch (choice)
15    {
16        case 'A': cout << "You entered A.\n";
17        case 'B': cout << "You entered B.\n";
18        case 'C': cout << "You entered C.\n";
19        default: cout << "You did not enter A, B, or C!\n";
20    }
21    return 0;
22 }
```

Program Output with Example Input Shown in Bold

```
Enter A, B, or C: A 
You entered A.
You entered B.
You entered C.
You did not enter A, B, or C!
```

Program Output with Example Input Shown in Bold

```
Enter A, B, or C: C 
You entered C.
You did not enter A, B, or C!
```

بدون عبارة 'break'، ينتقل البرنامج إلى جميع العبارات الموجودة أسفل العبارة التي تحتوي على تعبير 'case' المطابق. في بعض الأحيان، يكون هذا هو السلوك المطلوب. يُظهر البرنامج 4-25 ميزات ثلاث نماذج من أجهزة التلفزيون التي يمكن للعميل الاختيار من بينها. النموذج 100 يحتوي على تحكم عن بعد. النموذج 200 يحتوي على تحكم عن بعد وصوت ستريو. النموذج 300 يحتوي على تحكم عن بعد، صوت ستريو، وإمكانية الصورة داخل الصورة. يستخدم البرنامج عبارة 'switch' مع حذف مقصود لعبارات 'break' لطباعة ميزات النموذج المحدد.

Program 4-25

```
1 // This program is carefully constructed to use the "fall through"
2 // feature of the switch statement.
3 #include <iostream>
4 using namespace std;
5
6 int main()
7 {
8     int modelNum; // Model number
9
10    // Get a model number from the user.
11    cout << "Our TVs come in three models:\n";
12    cout << "The 100, 200, and 300. Which do you want? ";
13    cin >> modelNum;
14 }
```


Program 4-25 (continued)

```

15 // Display the model's features.
16 cout << "That model has the following features:\n";
17 switch (modelNum)
18 {
19     case 300: cout << "\tPicture-in-a-picture.\n";
20     case 200: cout << "\tStereo sound.\n";
21     case 100: cout << "\tRemote control.\n";
22             break;
23     default: cout << "You can only choose the 100,";
24             cout << "200, or 300.\n";
25 }
26 return 0;
27 }

```

Program Output with Example Input Shown in Bold

Our TVs come in three models:
 The 100, 200, and 300. Which do you want? **100**
 That model has the following features:
 Remote control.

Program Output with Example Input Shown in Bold

Our TVs come in three models:
 The 100, 200, and 300. Which do you want? **200**
 That model has the following features:
 Stereo sound.
 Remote control.

Program Output with Example Input Shown in Bold

Our TVs come in three models:
 The 100, 200, and 300. Which do you want? **300**
 That model has the following features:
 Picture-in-a-picture.
 Stereo sound.
 Remote control.

Program Output with Example Input Shown in Bold

Our TVs come in three models:
 The 100, 200, and 300. Which do you want? **500**
 That model has the following features:
 You can only choose the 100, 200, or 300.

مثال آخر على مدى فائدة هذه القدرة "الانتقال التلقائي" (fall through) هو عندما تريد أن ينتقل البرنامج إلى نفس مجموعة العبارات لعدة تعبيرات `case`. على سبيل المثال، يطلب البرنامج 4-26 من المستخدم اختيار درجة من درجات طعام الحيوانات الأليفة. الخيارات المتاحة هي A، B، وC. ستتعامل عبارة `switch` مع الأحرف الكبيرة (uppercase) أو الصغيرة (lowercase).

Program 4-26

```

1 // The switch statement in this program uses the "fall through"
2 // feature to catch both uppercase and lowercase letters entered
3 // by the user.
4 #include <iostream>

```



```

5  using namespace std;
6
7  int main()
8  {
9      char feedGrade;
10
11     // Get the desired grade of feed.
12     cout << "Our pet food is available in three grades:\n";
13     cout << "A, B, and C. Which do you want pricing for? ";
14     cin >> feedGrade;
15
16     // Display the price.
17     switch(feedGrade)
18     {
19         case 'a':
20         case 'A': cout << "30 cents per pound.\n";
21                 break;
22         case 'b':
23         case 'B': cout << "20 cents per pound.\n";
24                 break;
25         case 'c':
26         case 'C': cout << "15 cents per pound.\n";
27                 break;
28         default: cout << "That is an invalid choice.\n";
29     }
30     return 0;
31 }

```

Program Output with Example Input Shown in Bold

Our pet food is available in three grades:
A, B, and C. Which do you want pricing for? **b**
20 cents per pound.

Program Output with Example Input Shown in Bold

Our pet food is available in three grades:
A, B, and C. Which do you want pricing for? **B**
20 cents per pound.

Using switch in Menu Systems

عبارة 'switch' هي أداة طبيعية لبناء أنظمة القوائم (menu systems). تذكر أن البرنامج 4-18 يعرض قائمة لتحديد باقة النادي الصحي التي يرغب المستخدم في شرائها. يستخدم البرنامج عبارات 'if/else if' لتحديد الباقة التي اختارها المستخدم وعرض الرسوم المحسوبة. البرنامج 4-27 هو تعديل على ذلك البرنامج، حيث يستخدم عبارة 'switch' بدلاً من 'if/else if'.

Program 4-27

```
1 // This program uses a switch statement to determine
2 // the item selected from a menu.
3 #include <iostream>
4 #include <iomanip>
5 using namespace std;
6
7 int main()
8 {
9     int choice;        // To hold a menu choice
10    int months;        // To hold the number of months
11    double charges;    // To hold the monthly charges
12
13    // Constants for membership rates
14    const double ADULT = 40.0,
15                CHILD = 20.0,
16                SENIOR = 30.0;
17
18    // Constants for menu choices
19    const int ADULT_CHOICE = 1,
20            CHILD_CHOICE = 2,
21            SENIOR_CHOICE = 3,
22            QUIT_CHOICE = 4;
23
24    // Display the menu and get a choice.
25    cout << "\t\tHealth Club Membership Menu\n\n"
26         << "1. Standard Adult Membership\n"
27         << "2. Child Membership\n"
28         << "3. Senior Citizen Membership\n"
29         << "4. Quit the Program\n\n"
30         << "Enter your choice: ";
31    cin >> choice;
32
33    // Set the numeric output formatting.
34    cout << fixed << showpoint << setprecision(2);
35
36    // Respond to the user's menu selection.
37    switch (choice)
38    {
39        case ADULT_CHOICE:
40            cout << "For how many months? ";
41            cin >> months;
42            charges = months * ADULT;
43            cout << "The total charges are $" << charges << endl;
44            break;
45
46        case CHILD_CHOICE:
47            cout << "For how many months? ";
48            cin >> months;
49            charges = months * CHILD;
50            cout << "The total charges are $" << charges << endl;
51            break;
52    }
```



```

53         case SENIOR_CHOICE:
54             cout << "For how many months? ";
55             cin >> months;
56             charges = months * SENIOR;
57             cout << "The total charges are $" << charges << endl;
58             break;
59
60         case QUIT_CHOICE:
61             cout << "Program ending.\n";
62             break;
63
64         default:
65             cout << "The valid choices are 1 through 4. Run the\n"
66                 << "program again and select one of those.\n";
67     }
68
69     return 0;
70 }

```

Program Output with Example Input Shown in Bold

Health Club Membership Menu

1. Standard Adult Membership
2. Child Membership
3. Senior Citizen Membership
4. Quit the Program

Enter your choice: **2**
 For how many months? **6**
 The total charges are \$120.00

4.16 The switch Statement with Initialization

المفهوم: بدءًا من الإصدار C++17، يمكن أن تحتوي عبارة `switch` على بند تهيئة اختياري.

إذا كنت تستخدم الإصدار C++17 أو أحدث، يمكنك كتابة عبارة `switch` مع بند تهيئة اختياري يتم تنفيذه قبل تقييم التعبير الصحيح الشرطي. إليك الشكل العام:

```

switch (Initialization; IntegerExpression)
{
    case ConstantExpression:
        // place one or more statements here
    case ConstantExpression:
        // place one or more statements here
    // case statements may be repeated as many
    // times as necessary
    default:
        // place one or more statements here
}

```


لنلق نظرة على مثال. في الكود التالي، افترض أن number هو من نوع double:

```
switch (int value = abs(number); value)
{
    case 1:
        cout << "one" << endl;
        break;
    case 2:
        cout << "two" << endl;
        break;
    default:
        cout << "Invalid value" << endl;
        break;
}
```

في عبارة switch هذه، بند التهيئة (initialization clause) هو:

```
int value = abs(number);
```

يُعرّف بند التهيئة متغيراً من نوع 'int' يُسمى 'value' ويعين له القيمة المطلقة لـ 'number'. ثم يتم تقييم المتغير 'value'.

عندما يتم تعريف متغير في بند التهيئة لعبارة 'switch'، فإن نطاق هذا المتغير ('scope') يقتصر على عبارة 'switch' نفسها. على سبيل المثال، الكود التالي سيسبب خطأ:

```
switch (int value = abs(number); value)
{
    case 1:
        cout << "one" << endl;
        break;
    case 2:
        cout << "two" << endl;
        break;
    default:
        cout << "Invalid value" << endl;
        break;
}
// The following statement causes an error.
cout << value << endl;
```

لن يتم تجميع العبارة الأخيرة لأنها تحاول استخدام المتغير 'value'، الذي يمكن الوصول إليه فقط من خلال الكود الموجود داخل عبارة 'switch'.

4.17 More about Blocks and Variable Scope

المفهوم: نطاق المتغير يقتصر على الكتلة التي يتم تعريفه فيها.

تسمح لغة C++ بإنشاء المتغيرات في أي مكان تقريباً في البرنامج. البرنامج 4-28 هو تعديل للبرنامج 4-17، الذي يحدد ما إذا كان المستخدم مؤهلاً للحصول على قرض. تم نقل تعريفات المتغيرات `income` و `years` إلى نقاط لاحقة في البرنامج.

Program 4-28

```
1  // This program demonstrates late variable definition
2  #include <iostream>
3  using namespace std;
4
5  int main()
6  {
7      // Constants for minimum income and years
8      const double MIN_INCOME = 35000.0;
9      const int MIN_YEARS = 5;
10
11     // Get the annual income.
12     cout << "What is your annual income? ";
13     double income; // Variable definition
14     cin >> income;
15
16     // Get the number of years at the current job.
17     cout << "How many years have you worked at "
18          << "your current job? ";
19     int years; // Variable definition
20     cin >> years;
21
22     // Determine the user's loan qualifications.
23     if (income >= MIN_INCOME || years > MIN_YEARS)
24         cout << "You qualify.\n";
25     else
26     {
27         cout << "You must earn at least $"
28              << MIN_INCOME << " or have been "
29              << "employed more than " << MIN_YEARS
30              << " years.\n";
31     }
32     return 0;
33 }
```


من الممارسات الشائعة تعريف جميع متغيرات الدالة في أعلى الدالة. في بعض الأحيان، خاصة في البرامج الأطول، يُعد تعريف المتغيرات بالقرب من الجزء الذي يتم استخدامها فيه فكرة جيدة. هذا يجعل الغرض من المتغير أكثر وضوحًا.

تذكر من الفصل الثاني أن نطاق المتغير يُعرف على أنه الجزء من البرنامج حيث يمكن استخدام المتغير.

في البرنامج 4-28، نطاق متغير `income` هو الجزء من البرنامج في الأسطر من 13 إلى 32. نطاق متغير `years` هو الجزء من البرنامج في الأسطر من 19 إلى 32.

يتم تعريف المتغيرات `income` و `years` داخل الأقواس الخاصة بالدالة الرئيسية `main`. المتغيرات التي يتم تعريفها داخل مجموعة من الأقواس لها نطاق محلي أو نطاق كتلة. يمكن استخدامها فقط في الجزء من البرنامج بين تعريفها وقوس الإغلاق للكتلة.

يمكنك تعريف المتغيرات داخل أي كتلة. على سبيل المثال، انظر إلى البرنامج 4-29. هذا الإصدار من برنامج القرض يحتوي على المتغير `years` المحدد داخل كتلة عبارة `if`. نطاق `years` هو الجزء من البرنامج في الأسطر من 21 إلى 31.

Program 4-29

```
1 // This program demonstrates a variable defined in an inner block.
2 #include <iostream>
3 using namespace std;
4
5 int main()
6 {
7     // Constants for minimum income and years
8     const double MIN_INCOME = 35000.0;
9     const int MIN_YEARS = 5;
10
11     // Get the annual income.
12     cout << "What is your annual income? ";
13     double income; // Variable definition
```

Program 4-29 (continued)

```
14     cin >> income;
15
16     if (income >= MIN_INCOME)
17     {
18         // Get the number of years at the current job.
19         cout << "How many years have you worked at "
20             << "your current job? ";
21         int years; // Variable definition
22         cin >> years;
```



```

23
24         if (years > MIN_YEARS)
25             cout << "You qualify.\n";
26         else
27         {
28             cout << "You must have been employed for\n"
29                 << "more than " << MIN_YEARS
30                 << " years to qualify.\n";
31         }
32     }
33     else
34     {
35         cout << "You must earn at least $" << MIN_INCOME
36             << " to qualify.\n";
37     }
38     return 0;
39 }

```

لاحظ أن نطاق `years` يقتصر فقط على الكتلة التي يتم تعريفه فيها. المتغير غير مرئي قبل تعريفه أو بعد قوس الإغلاق للكتلة. هذا ينطبق على أي متغير يتم تعريفه داخل مجموعة من الأقواس.



NOTE: When a program is running and it enters the section of code that constitutes a variable's scope, it is said that the variable *comes into scope*. This simply means the variable is now visible and the program may reference it. Likewise, when a variable *leaves scope*, or *goes out of scope*, it may no longer be used.

Variables with the Same Name

عندما تكون كتلة متداخلة داخل كتلة أخرى، يمكن أن يكون للمتغير المحدد في الكتلة الداخلية نفس اسم المتغير المحدد في الكتلة الخارجية. ومع ذلك، طالما أن المتغير في الكتلة الداخلية مرئي، فإن المتغير في الكتلة الخارجية سيتم إخفاؤه. يوضح هذا البرنامج 4-30.

Program 4-30

```
1 // This program uses two variables with the name number.
2 #include <iostream>
3 using namespace std;
4
5 int main()
6 {
7     // Define a variable named number.
8     int number;
9
10    cout << "Enter a number greater than 0: ";
11    cin >> number;
12    if (number > 0)
13    {
14        int number; // Another variable named number.
15        cout << "Now enter another number: ";
16        cin >> number;
17        cout << "The second number you entered was "
18             << number << endl;
19    }
20    cout << "Your first number was " << number << endl;
21    return 0;
22 }
```

Program Output with Example Input Shown in Bold

```
Enter a number greater than 0: 2 
Now enter another number: 7 
The second number you entered was 7
Your first number was 2
```

يحتوي البرنامج 4-30 على متغيرين منفصلين يحملان الاسم 'number'. تعمل عبارات 'cin' و 'cout' في الكتلة الداخلية (التابعة لعبارة 'if') فقط مع المتغير 'number' المحدد في تلك الكتلة. بمجرد مغادرة البرنامج لتلك الكتلة، يخرج المتغير الداخلي 'number' من النطاق، مما يكشف عن المتغير الخارجي 'number'.



WARNING! Although it's perfectly acceptable to define variables inside nested blocks, you should avoid giving them the same names as variables in the outer blocks. It's too easy to confuse one variable with another.

انتهت المحاضرة الرابعة